# SOFTWARE REQUIREMENT SPECIFICATION

## for

# LKFORUM WEBSITE

**Version 1.0**

**Prepared by : Vo An Khoi (23520790)**

**Submitted to : Nguyen Trinh Đong**
**Lecturer**

**May 8, 2026**

# Contents

# 1 Introduction

## 1.1 Purpose

This SRS describes the functional and nonfunctional requirements for software release 1.0 of the Community Social Network Website (LKForum). This document is intended to be used by the members of the project team who will implement and verify the correct functioning of the system. Unless otherwise noted, all requirements specified here are committed for release 1.0

## 1.2 Intended Audience and Reading Suggestions

This SRS is for developers, project managers, users and testers. Further the discussion will provide all the internal, external, functional and also non-functional informations about "LKFORUM WEBSITE".

## 1.3 Project Scope

This SRS describes the functional and nonfunctional requirements for software release 1.0 of the Community Social Network Website (LKForum). This document is intended to be used by the members of the project team who will implement and verify the correct functioning of the system. Unless otherwise noted, all requirements specified here are committed for release 1.0

## 1.4 References

# 2 Overall Description

## 2.1 Product Perspective

Community Social Network LKForum is a new website that provides a comprehensive, modern solution for building and managing online communities. The context diagram in Figure 1 illustrates the external entities and system interfaces for release 1.0. The system is expected to evolve over several releases, ultimately connecting to third-party social media APIs for authentication (Google, Facebook) and expanding to include advanced content monetization tools and mobile-native applications.

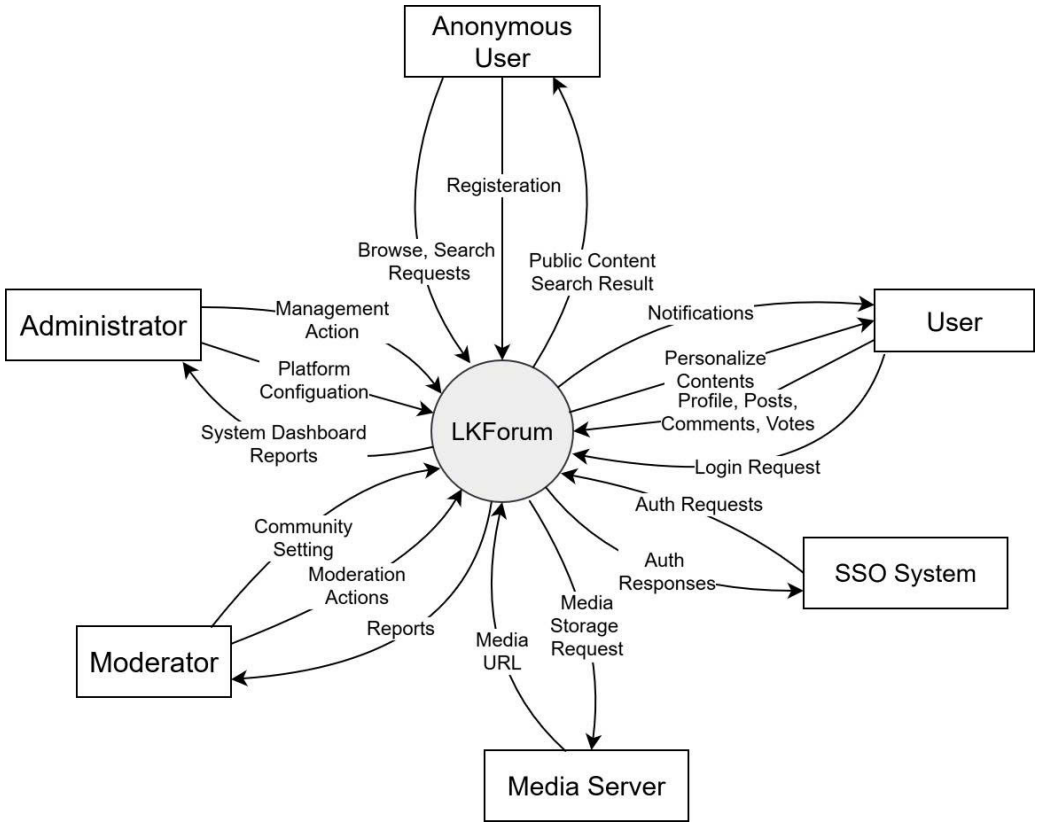


Figure 2.1: Context diagram for release 1.0 of LKForum

## 2.2 Operating Environment

**OE-1:** LKForum must operate correctly on modern web browsers including: Google Chrome (latest version), Mozilla Firefox (latest version), Apple Safari (latest version), and Microsoft Edge (latest version).

**OE-2:** Users can access LKForum via the Internet from desktop computers, laptops, and tablets. The interface must have responsive design to be compatible with various screen sizes.

## 2.3 Design and Implementation Constraints

**CO-1:** The user interface (front-end) source code must be developed using the Svelte framework.

**CO-2:** The system must use a standard MongoDB NoSQL database with a back-end developed using the Go framework.

**CO-3:** All HTML code must adhere to the HTML 5.0 standard.

## 2.4 Assumptions and Dependencies

**AS-1:** It is assumed that users have a stable Internet connection to access and use the platform's features.

**DE-1:** The social media login feature (Google, Facebook) depends on the APIs and policies of these third-party service providers. Any changes on their part may affect the operation of this feature.

# 3 System Features

The LKForum website is being built using the Go programming language for the back-end, Svelte with Vite for the front-end, MongoDB for primary data storage, and Redis for caching and session management.
Back-End – Go with the Gin web framework.
Front-End – Svelte with Vite.
Database – MongoDB.
Cache and Session Store – Redis.

## 3.1 Use Case Description

### 3.1.1 UC-01 - Sign Up

| | |
|---|---|
| **Name** | Sign Up |
| **Description** | This use case describes how a new user creates an account and verifies the email address using OTP. |
| **Actor** | User |
| **Trigger** | When the user clicks the “Sign Up” button. |
| **Pre-condition** | The user is not logged in to the system.<br>The user is on the sign up page. |
| **Post-condition** | The user account is created and verified.<br>The user is authenticated and can access the system. |

Table 3.1: Use Case Specification - Sign Up

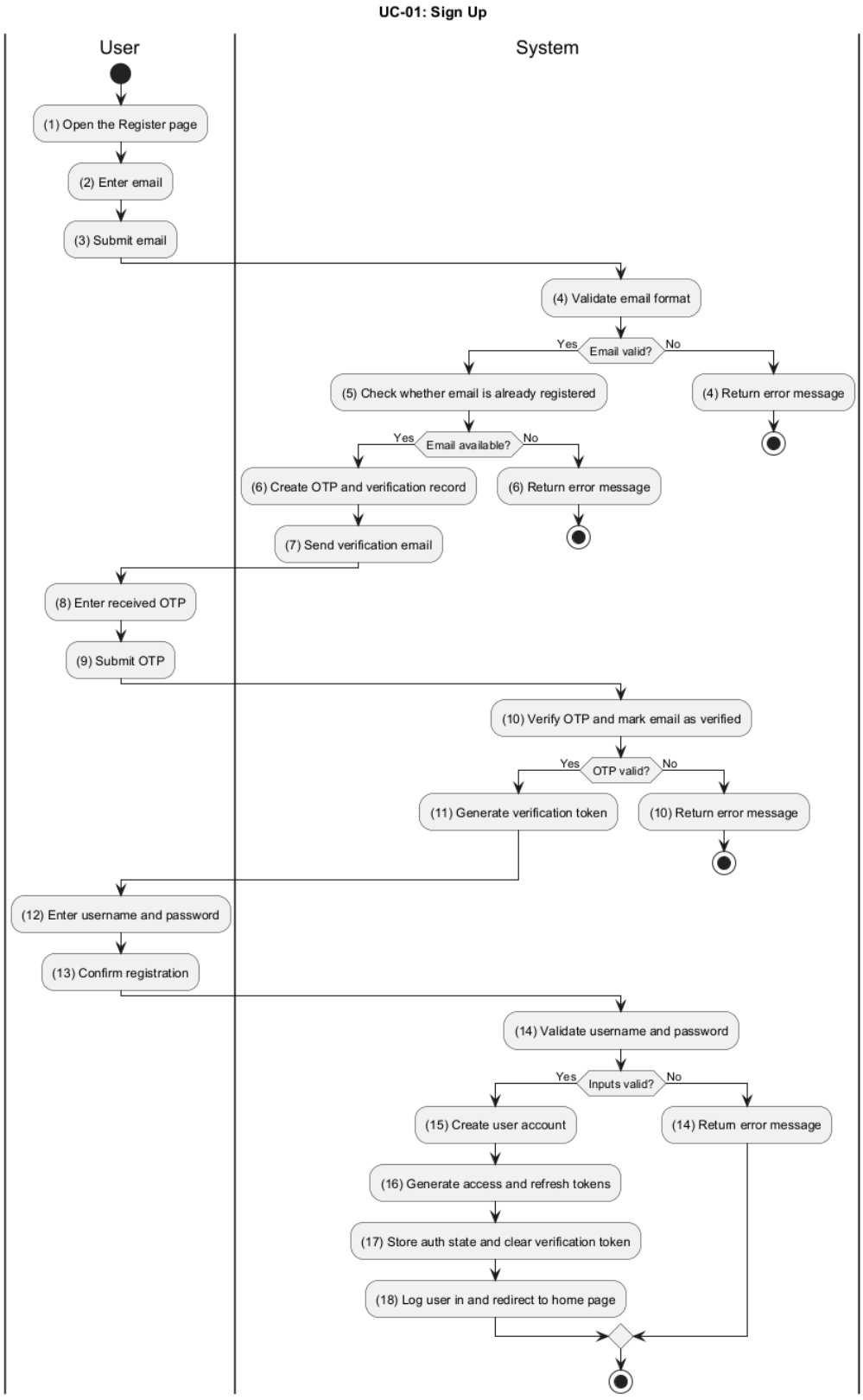


Figure 3.1: Activity Diagram for UC-01

| Activity | BR Code | Description |
|---|---|---|
| (4) | BR1 | **Validate Email Format:**<br>• The system checks that **email** is present and valid.<br>• If the **email** is missing or invalid, the system returns a bad-request error. |
| (5–7) | BR2 | **Create and Send Verification OTP:**<br>• The system checks whether the **email** already exists in the user repository.<br>• If the **email** already exists, the system returns an email-exists error.<br>• Otherwise, the system deletes any existing verification record for the same **email**.<br>• The system generates a 6-digit **OTP**, a **nonce**, and an expiry timestamp using ***generateOTP()*** and related helpers.<br>• The system stores a new email-verification record with **IsVerified = false**.<br>• If an email sender is configured, the system sends the **OTP** by email asynchronously. |
| (10–11) | BR3 | **Verify OTP and Issue Token:**<br>• The system checks that **email** and **otp** are present.<br>• If no verification record exists, the system returns an invalid **OTP** error.<br>• If the verification record is already marked verified, the system returns an email-already-verified error.<br>• If the **OTP** does not match or has expired, the system returns an invalid or expired **OTP** error.<br>• Otherwise, the system marks the verification record as verified.<br>• After successful **OTP** verification, the system generates a verification token from the verified **email** and **nonce**.<br>• The token is returned to the client and is required to complete registration. |

Table 3.2: Business Rules for UC-01 - Sign Up

| Activity | BR Code | Description |
|---|---|---|
| (14) | BR4 | **Validate Registration Fields:**<br>• The system checks that **username** is between 3 and 20 characters.<br>• The system checks that **password** is at least 8 characters long.<br>• The system checks that the **password** is strong enough, including uppercase, lowercase, number, and special character, using ***isStrongPassword()***.<br>• If any validation fails, the system returns the corresponding error message. |
| (15–18) | BR5 | **Create Account and Complete Login:**<br>• The system hashes the **password** with ***GenerateFromPassword()*** before storing it.<br>• The system checks that the **username** is still available before creating the account.<br>• The system checks again that the **email** is still not registered to prevent race conditions.<br>• The system creates the new local user with default settings and **IsVerified = true**.<br>• The system initializes the user role content and activity stats.<br>• The system deletes the email verification record after successful account creation.<br>• The system invalidates the **username** cache.<br>• The system generates access and refresh tokens using ***GenerateToken()*** and returns the auth response. |

Table 3.2: Business Rules for UC-01 - Sign Up

### 3.1.2 UC-02 - Sign In

| | |
|---|---|
| **Name** | Sign In |
| **Description** | This use case describes how a registered user logs into the system using local credentials or Google OAuth. |
| **Actor** | User |
| **Trigger** | When the user clicks the “Sign In” button. |
| **Pre-condition** | The user is not logged in to the system.<br>The user is on the sign in page. |
| **Post-condition** | The user is authenticated.<br>The user is redirected to the appropriate page. |

Table 3.3: Use Case Specification - Sign In

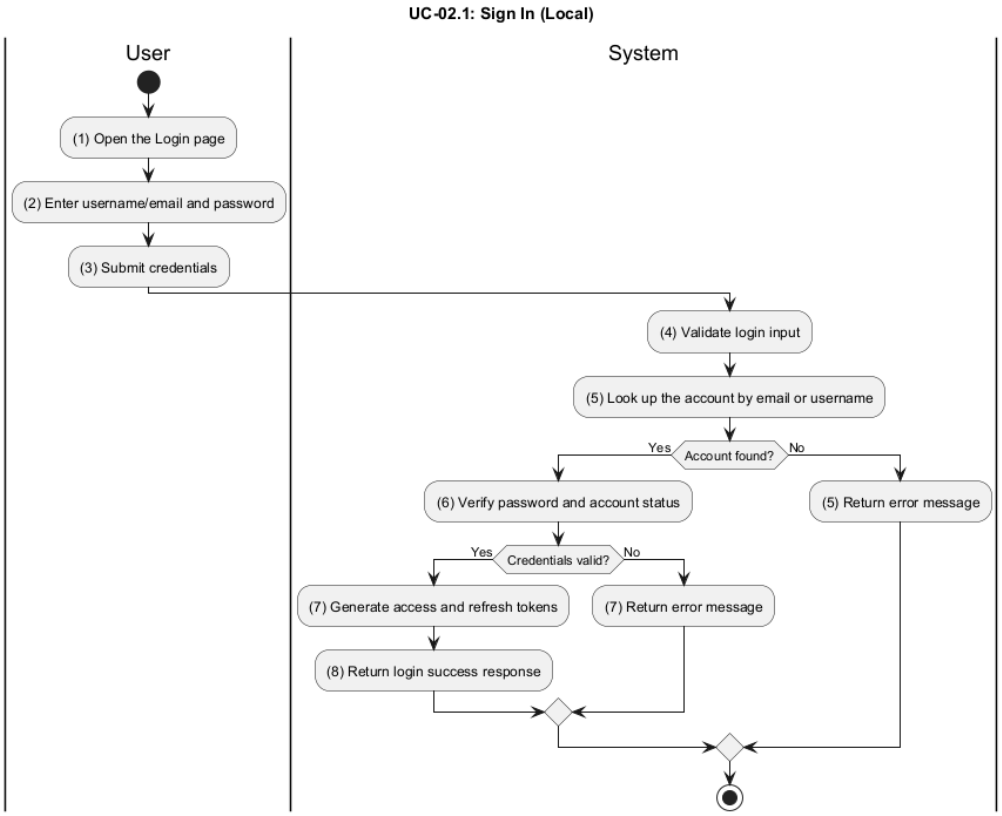


Figure 3.2: Activity Diagram for UC-02 Local Sign In

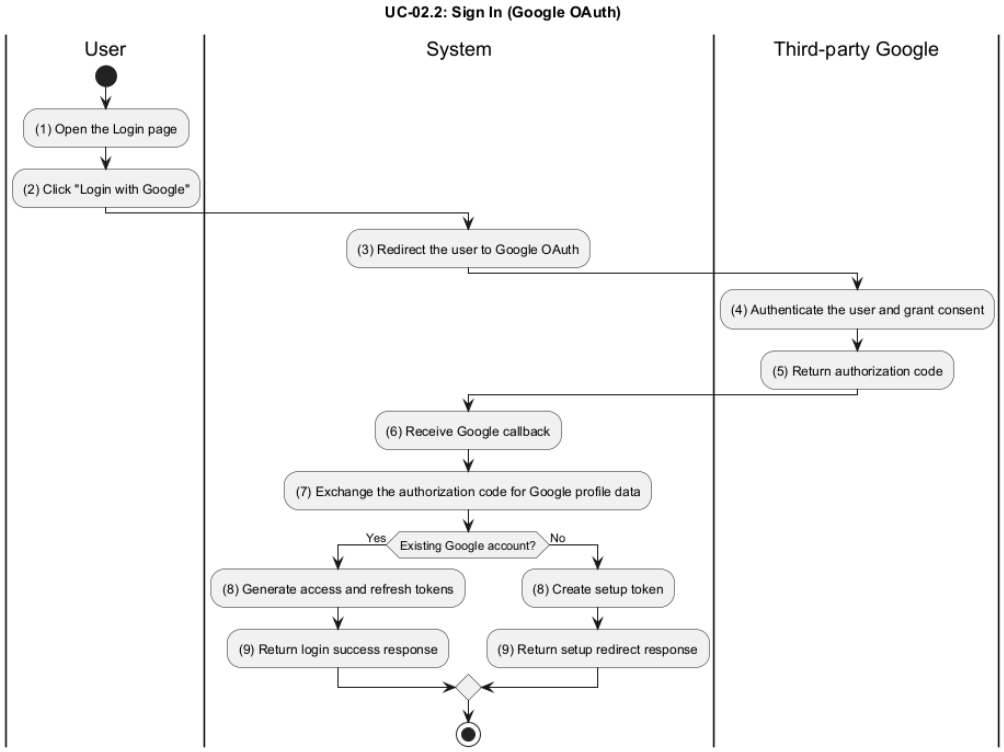


Figure 3.3: Activity Diagram for UC-02 Google Sign In

### 3.1.3 UC-03 - Forgot Password

| | |
|---|---|
| **Name** | Forgot Password |
| **Description** | This use case describes how a user resets a forgotten password through email verification and OTP. |
| **Actor** | User |
| **Trigger** | When the user clicks the “Forgot Password” link. |
| **Pre-condition** | The user has an existing account and can access the registered email address.<br>The user is not logged in to the system. |
| **Post-condition** | The user password is reset successfully.<br>The user can sign in again with the new password. |

Table 3.4: Use Case Specification - Forgot Password

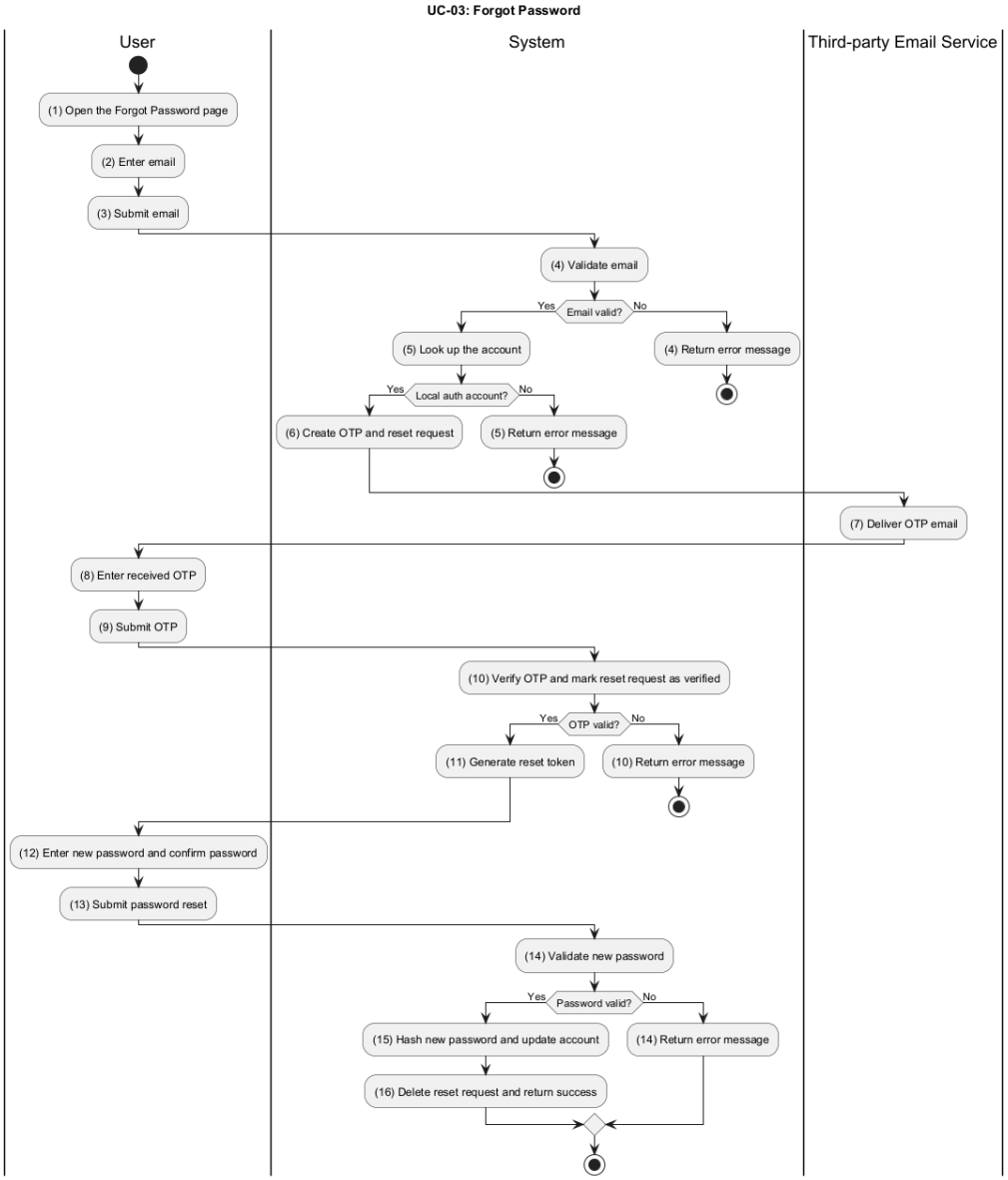


Figure 3.4: Activity Diagram for UC-03

| Activity | BR Code | Description |
|---|---|---|
| (4) | BR1 | **Validate Reset Request:**<br>• The system checks that **email** is present and valid.<br>• The system looks up the account by **email** using ***GetByEmail()***.<br>• If the account does not exist, the system returns an email-not-registered error.<br>• If the account exists but the **provider** is not local, the system returns a login-method-mismatch error. |
| (5–7) | BR2 | **Create Password Reset Request:**<br>• The system deletes any existing reset record for the same **email** so the user can request a new OTP.<br>• The system generates a 6-digit **OTP**, a **nonce**, and an expiry timestamp using ***generateOTP()*** and ***generateNonce()***.<br>• The system stores a new password-reset record with **IsVerified = false**.<br>• If an email sender is configured, the system sends the reset OTP asynchronously using ***SendPasswordResetEmail()***. |
| (10–11) | BR3 | **Verify Reset OTP and Issue Token:**<br>• The system checks that **email** and **otp** are present.<br>• If no reset record exists, the system returns an invalid **OTP** error.<br>• If the reset record is already verified, the system returns an email-already-verified error.<br>• If the **OTP** does not match or has expired, the system returns an invalid or expired **OTP** error.<br>• Otherwise, the system marks the reset record as verified.<br>• The system generates a reset token from the verified **email** and **nonce** using ***CreateVerificationToken()***. |

Table 3.5: Business Rules for UC-03 - Forgot Password

| Activity | BR Code | Description |
|---|---|---|
| (14–16) | BR4 | **Reset Password:**<br>• The system checks that **newPassword** is at least 8 characters long.<br>• The system checks that **newPassword** is strong enough using ***isStrongPassword()***.<br>• The system parses the reset token using ***auth.ParseVerificationToken()*** and verifies the stored **nonce**.<br>• The system hashes **newPassword** with ***GenerateFromPassword()*** and updates the user account.<br>• The system deletes the reset record after a successful password change. |

Table 3.5: Business Rules for UC-03 - Forgot Password

### 3.1.4 UC-04 - Manage Profile

| | |
|---|---|
| **Name** | Manage Profile |
| **Description** | This use case describes how a user updates profile information such as bio, avatar, and cover image. |
| **Actor** | User |
| **Trigger** | When the user opens the profile edit page. |
| **Pre-condition** | The user is logged in to the system.<br>The user has permission to edit the profile. |
| **Post-condition** | The profile information is updated and stored.<br>The new profile data is visible across the system. |

Table 3.6: Use Case Specification - Manage Profile

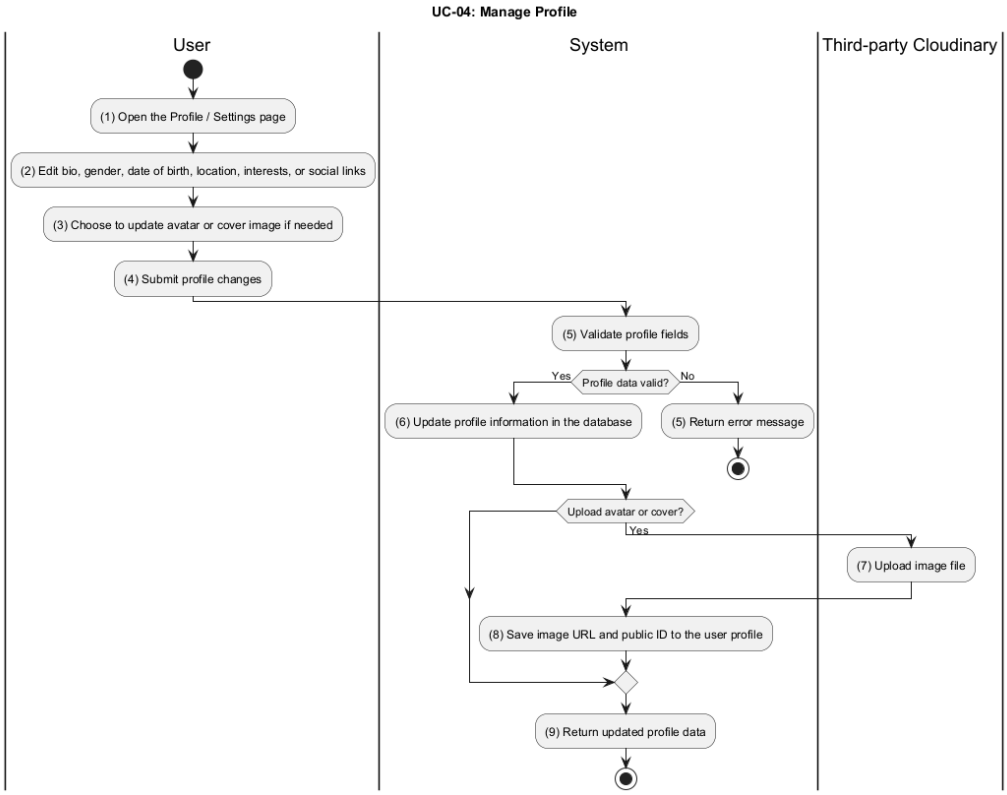


Figure 3.5: Activity Diagram for UC-04

| Activity | BR Code | Description |
|---|---|---|
| (5) | BR1 | **Validate Profile Fields:**<br>• The system checks the submitted profile fields before saving.<br>• **bio** is limited by DTO validation, **gender** must match a valid model value, and **date_of_birth** must parse as a valid date.<br>• The system validates **location** against allowed provinces and ensures **interests** do not exceed the maximum allowed count. |
| (6) | BR2 | **Update Profile Data:**<br>• The system loads the current user and updates **RoleContent.AsUser**.<br>• Empty string values clear the corresponding profile field.<br>• Invalid **gender**, **location**, or **interest** values return an error instead of being saved. |

Table 3.7: Business Rules for UC-04 - Manage Profile

| Activity | BR Code | Description |
|---|---|---|
| (7–8) | BR3 | **Upload and Save Profile Images:**<br>• When the user uploads an avatar or cover image, the system sends the file to ***cloudinary.UploadImages()***.<br>• The system stores the returned image **URL** and **PublicID** through ***UpdateAvatar()*** or ***UpdateCover()***.<br>• If an older image exists, the system deletes the old asset asynchronously using ***cloudinary.Delete()***. |
| (9) | BR4 | **Return Updated Profile:**<br>• The system returns the refreshed user profile after saving changes. |

Table 3.7: Business Rules for UC-04 - Manage Profile

### 3.1.5 UC-05 - Change Settings

| | |
|---|---|
| **Name** | Change Settings |
| **Description** | This use case describes how a user changes account preferences such as privacy, notifications, and theme. |
| **Actor** | User |
| **Trigger** | When the user opens the settings page. |
| **Pre-condition** | The user is logged in to the system.<br>The settings page is available to the user. |
| **Post-condition** | The selected preferences are saved.<br>The account settings are updated for future sessions. |

Table 3.8: Use Case Specification - Change Settings

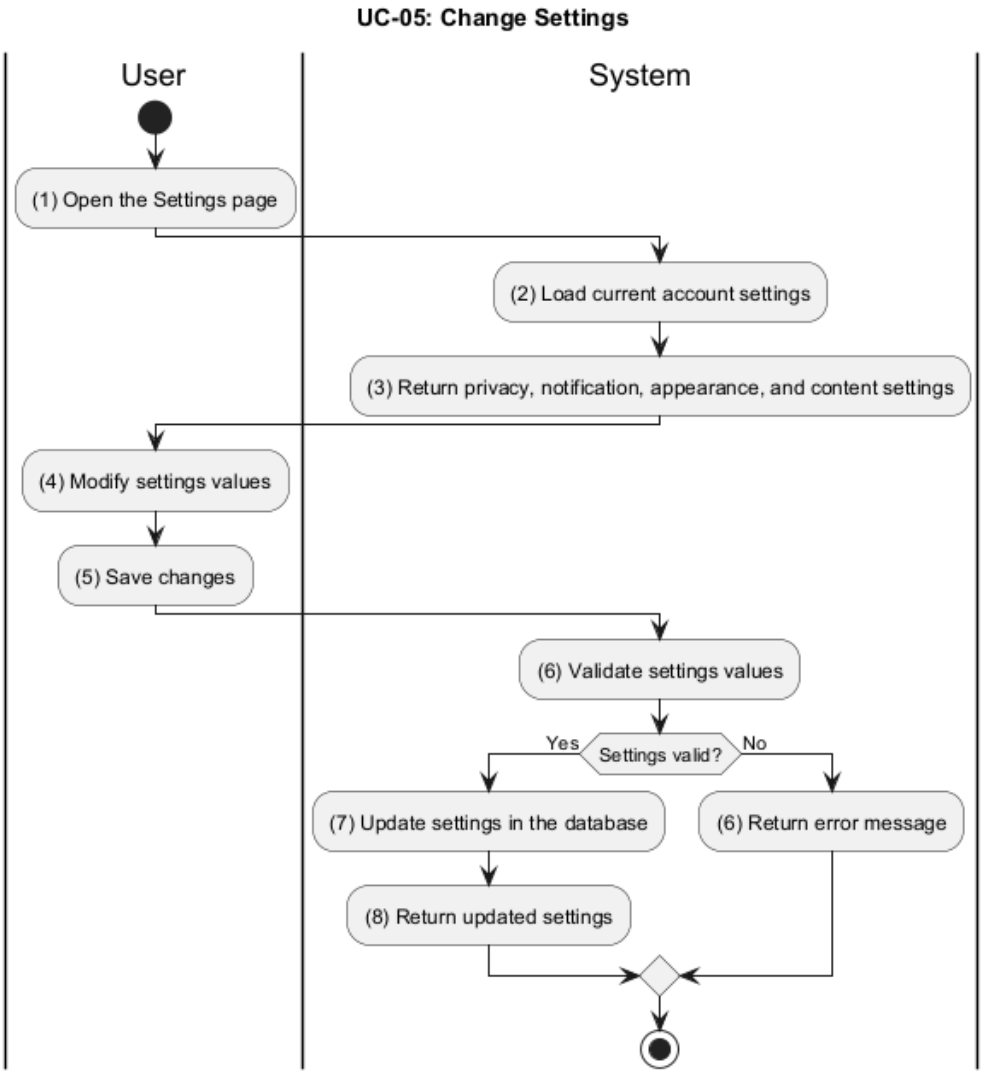


Figure 3.6: Activity Diagram for UC-05

| Activity | BR Code | Description |
|---|---|---|
| (2–3) | BR1 | **Load Current Settings:**<br>• The system loads the user's current **Settings** from the database using ***GetSettings()***.<br>• If the user has no settings record, the system returns default settings through ***dto.FromSettings()***. |

Table 3.9: Business Rules for UC-05 - Change Settings

| Activity | BR Code | Description |
|---|---|---|
| (6) | BR2 | **Validate Settings Values:**<br>• The system validates **theme** and **fontSize** using ***model.IsValidTheme()*** and ***model.IsValidFontSize()***.<br>• The system accepts only values supported by the model enums and returns a bad-request error for invalid input. |
| (7–8) | BR3 | **Persist Updated Settings:**<br>• The system updates **Appearance**, **Notifications**, **Privacy**, and **Content** fields in **user.Settings**.<br>• The system saves the updated user record with ***userRepo.Update()***.<br>• The system returns the updated settings after persistence. |

Table 3.9: Business Rules for UC-05 - Change Settings

### 3.1.6 UC-06 - View Activity History

| | |
|---|---|
| **Name** | View Activity History |
| **Description** | This use case describes how a user reviews past posts, comments, and interactions. |
| **Actor** | User |
| **Trigger** | When the user opens the activity history page. |
| **Pre-condition** | The user is logged in to the system.<br>The user has activity records in the system. |
| **Post-condition** | The activity history is displayed to the user.<br>The user can review prior activity. |

Table 3.10: Use Case Specification - View Activity History

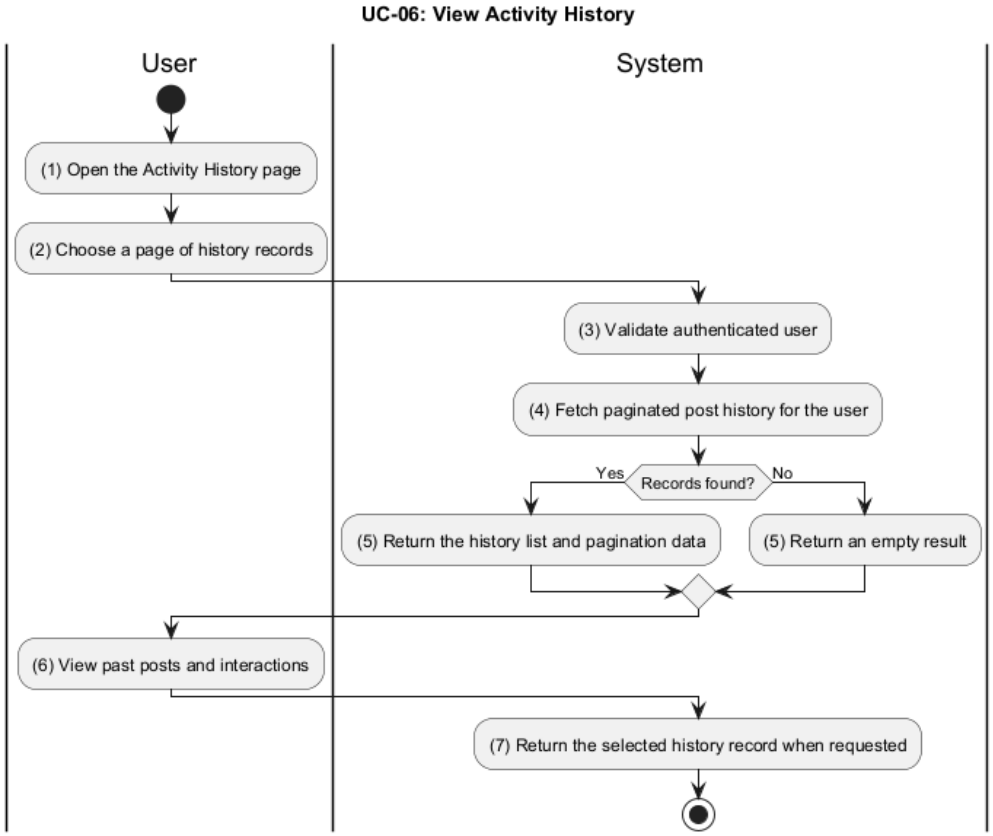


Figure 3.7: Activity Diagram for UC-06

| Activity | BR Code | Description |
|---|---|---|
| (3–5) | BR1 | **Validate Ownership and Fetch History:**<br>• The system checks that the requester is authenticated and that **userID** matches the current user for the history request.<br>• The system fetches paginated post history using ***GetPostHistoryByUserID()***.<br>• If no records are found, the system returns an empty result with pagination data. |
| (7) | BR2 | **Return Selected History Record:**<br>• The system returns a single history item only when the requester owns the record.<br>• The system rejects access if **requesterID** does not match the stored **UserID**. |

Table 3.11: Business Rules for UC-06 - View Activity History

### 3.1.7 UC-07 - Follow / Unfollow User

| | |
|---|---|
| **Name** | Follow / Unfollow User |
| **Description** | This use case describes how a user subscribes to or unsubscribes from another user's activity feed. |
| **Actor** | User |
| **Trigger** | When the user clicks the follow or unfollow action on another profile. |
| **Pre-condition** | The user is logged in to the system.<br>The target profile is available for follow actions. |
| **Post-condition** | The follow relationship is created or removed.<br>The user's feed and follower lists are updated. |

Table 3.12: Use Case Specification - Follow / Unfollow User

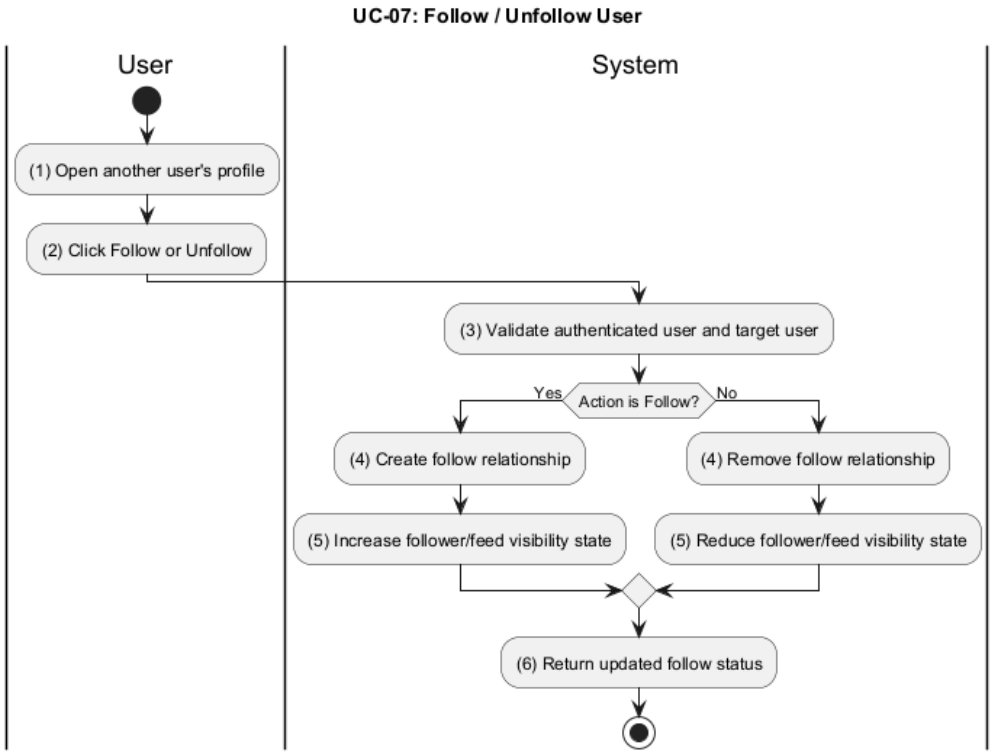


Figure 3.8: Activity Diagram for UC-07

| Activity | BR Code | Description |
|---|---|---|
| (3–6) | BR1 | **Backend Rule Status:**<br>• No follow or unfollow route, controller, or service is present in the backend code.<br>• The repository currently has no server-side business rules to define for this use case.<br>• This use case remains documented-only until a backend implementation is added. |

Table 3.13: Business Rules for UC-07 - Follow / Unfollow User

### 3.1.8 UC-08 - Browse Content

| | |
|---|---|
| **Name** | Browse Content |
| **Description** | This use case describes how a visitor or user explores communities and public posts. |
| **Actor** | Guest User / User |
| **Trigger** | When the user opens the forum homepage or a community page. |
| **Pre-condition** | The browsing page is accessible.<br>The user may be logged in or not logged in. |
| **Post-condition** | Public posts and community content are shown.<br>The user can continue browsing or open a post. |

Table 3.14: Use Case Specification - Browse Content

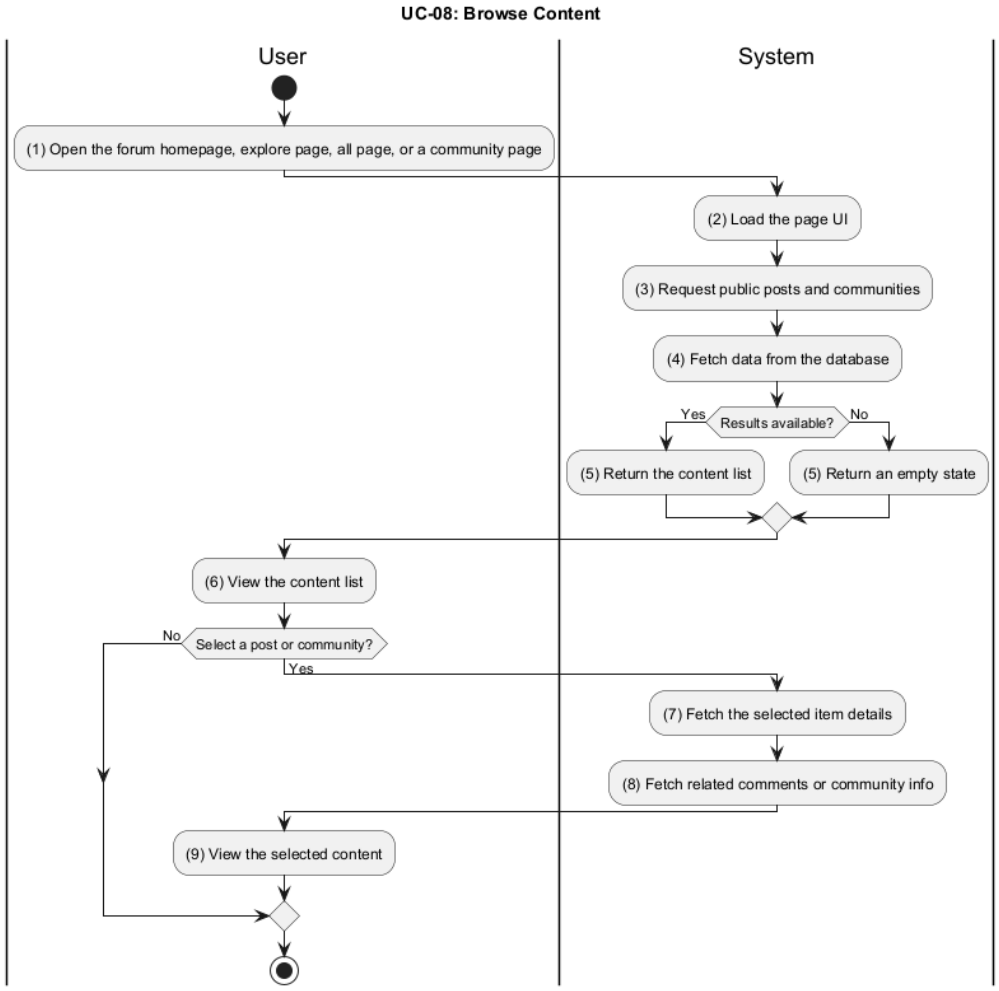


Figure 3.9: Activity Diagram for UC-08

| **Activity** | **BR Code** | **Description** |
|---|---|---|
| (2–5) | BR1 | **Load Public Content:**<br>• The system loads the page UI and requests public posts and communities.<br>• The system fetches content from the database using ***GetPosts()*** and community lookup functions.<br>• If no results exist, the system returns an empty state instead of an error. |

Table 3.15: Business Rules for UC-08 - Browse Content

| Activity | BR Code | Description |
|---|---|---|
| (7–9) | BR2 | **Open Selected Content:**<br>• If the user selects a post, the system loads the post details using ***GetPostByID()***.<br>• If the user selects a community, the system loads the community details using ***GetCommunityByID()*** or ***GetCommunityByName()***.<br>• The system also returns related comments or community information for the selected item. |

Table 3.15: Business Rules for UC-08 - Browse Content

### 3.1.9 UC-09 - Search Content

| | |
|---|---|
| **Name** | Search Content |
| **Description** | This use case describes how a user searches for posts, users, or communities by keyword. |
| **Actor** | User |
| **Trigger** | When the user enters a search term and submits the search. |
| **Pre-condition** | The search page is available.<br>The system has searchable content. |
| **Post-condition** | Matching results are returned to the user.<br>The user can open a result or refine the search. |

Table 3.16: Use Case Specification - Search Content

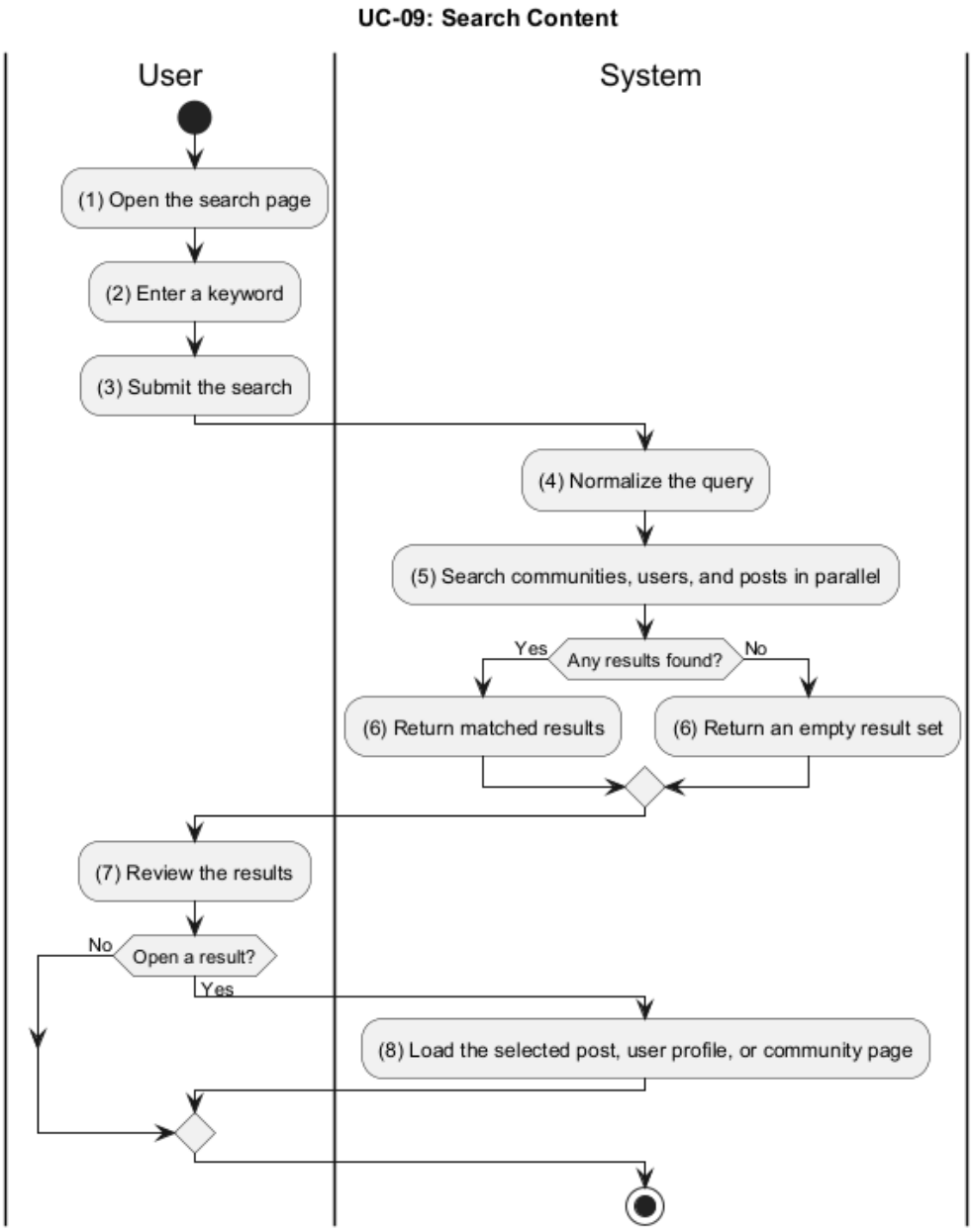


Figure 3.10: Activity Diagram for UC-09

| Activity | BR Code | Description |
|---|---|---|
| (4–6) | BR1 | **Normalize and Search in Parallel:**<br>• The system normalizes the query before searching.<br>• The system searches posts, users, and communities in parallel using ***GetPosts()***, ***GetUsers()***, and ***GetCommunitiesFilter()***.<br>• If no results are found, the system returns an empty result set. |
| (8) | BR2 | **Open Search Result:**<br>• If the user opens a result, the system loads the selected post, user profile, or community page.<br>• The system uses the corresponding detail lookup function based on the result type. |

Table 3.17: Business Rules for UC-09 - Search Content

### 3.1.10 UC-10 - Create Post

| | |
|---|---|
| **Name** | Create Post |
| **Description** | This use case describes how a member creates and submits a new post in a community. |
| **Actor** | User |
| **Trigger** | When the user clicks the “Create Post” action. |
| **Pre-condition** | The user is logged in to the system.<br>The user is a member of the selected community. |
| **Post-condition** | The post is stored successfully.<br>The post appears in the community or enters moderation if required. |

Table 3.18: Use Case Specification - Create Post

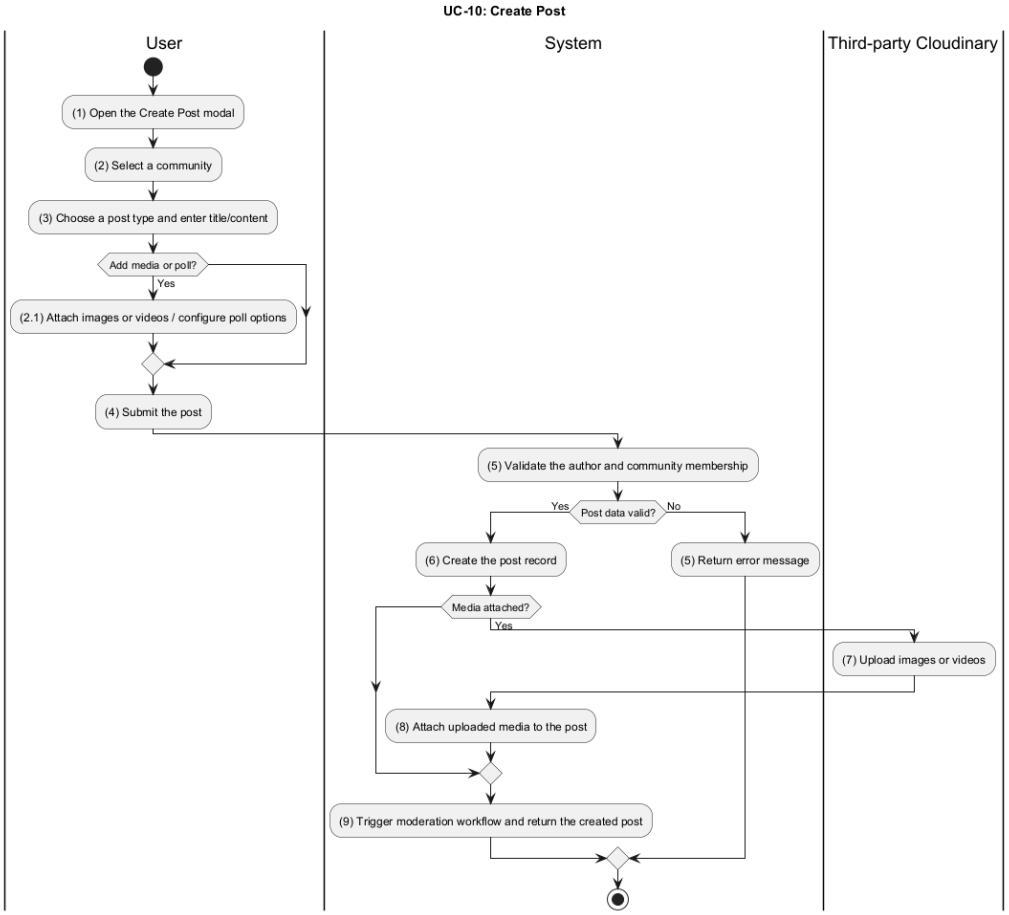


Figure 3.11: Activity Diagram for UC-10

| Activity | BR Code | Description |
|---|---|---|
| (5) | BR1 | **Validate Author and Community:**<br>• The system checks that the user is authenticated and is a member of the selected community.<br>• The system rejects the request if the community is banned or the user is banned from the community. |
| (6) | BR2 | **Create the Post Record:**<br>• The system creates the post with the submitted **title**, **content**, **tags**, and **type**.<br>• The system sets the initial moderation state based on community approval rules and author role.<br>• The system auto-upvotes the post for the creator using ***VoteOnTargetByAuthor()***. |

Table 3.19: Business Rules for UC-10 - Create Post

| Activity | BR Code | Description |
|---|---|---|
| (7–8) | BR3 | **Handle Media and Poll Data:**<br>• If images or videos are attached, the system uploads them using ***UploadImages()*** or ***UploadVideos()***.<br>• If a poll is included, the system stores poll options and poll settings in the post content. |
| (9) | BR4 | **Return Created Post:**<br>• The system returns the created post after persistence and moderation setup are complete. |

Table 3.19: Business Rules for UC-10 - Create Post

### 3.1.11 UC-11 - Edit Post

| | |
|---|---|
| **Name** | Edit Post |
| **Description** | This use case describes how a member modifies an existing post and preserves edit history. |
| **Actor** | User |
| **Trigger** | When the user opens an existing post and chooses “Edit”. |
| **Pre-condition** | The user is logged in to the system.<br>The user owns the post or has edit permission. |
| **Post-condition** | The edited post is saved successfully.<br>The edit history is retained. |

Table 3.20: Use Case Specification - Edit Post

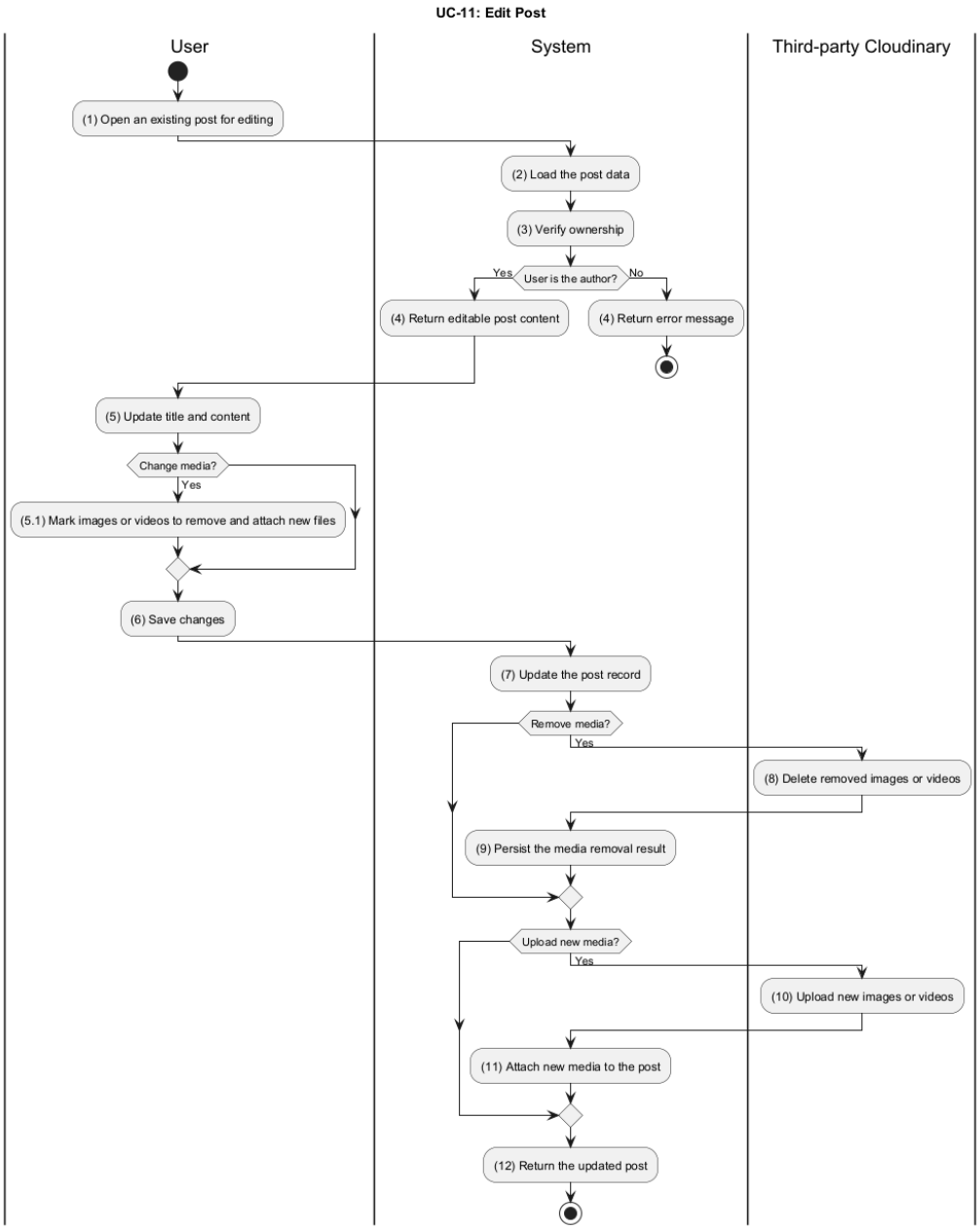


Figure 3.12: Activity Diagram for UC-11

| Activity | BR Code | Description |
|---|---|---|
| (2–4) | BR1 | **Authorize Editing:**<br>• The system loads the post and verifies that the requester is the author.<br>• If the requester is not the author, the system returns a forbidden error. |

Table 3.21: Business Rules for UC-11 - Edit Post

| Activity | BR Code | Description |
|---|---|---|
| (5–6) | BR2 | **Apply Content Changes:**<br>• The system updates **title**, **text**, and **tags** when they are provided.<br>• If no fields change, the system returns the current post without modifying data. |
| (7–11) | BR3 | **Handle Media Updates:**<br>• If media is removed, the system deletes the old assets using ***Delete()***.<br>• If new media is uploaded, the system stores it using ***UploadImages()*** or ***UploadVideos()***.<br>• The system persists the updated media links back to the post record. |
| (12) | BR4 | **Return Updated Post:**<br>• The system returns the refreshed post after the update is saved. |

Table 3.21: Business Rules for UC-11 - Edit Post

### 3.1.12 UC-12 - Delete Post

| | |
|---|---|
| **Name** | Delete Post |
| **Description** | This use case describes how a member or moderator removes a post from public view using soft delete. |
| **Actor** | User / Moderator |
| **Trigger** | When the user clicks the “Delete” action on a post. |
| **Pre-condition** | The user is logged in to the system.<br>The user has permission to delete the post. |
| **Post-condition** | The post is marked as deleted.<br>The post is hidden from public view. |

Table 3.22: Use Case Specification - Delete Post

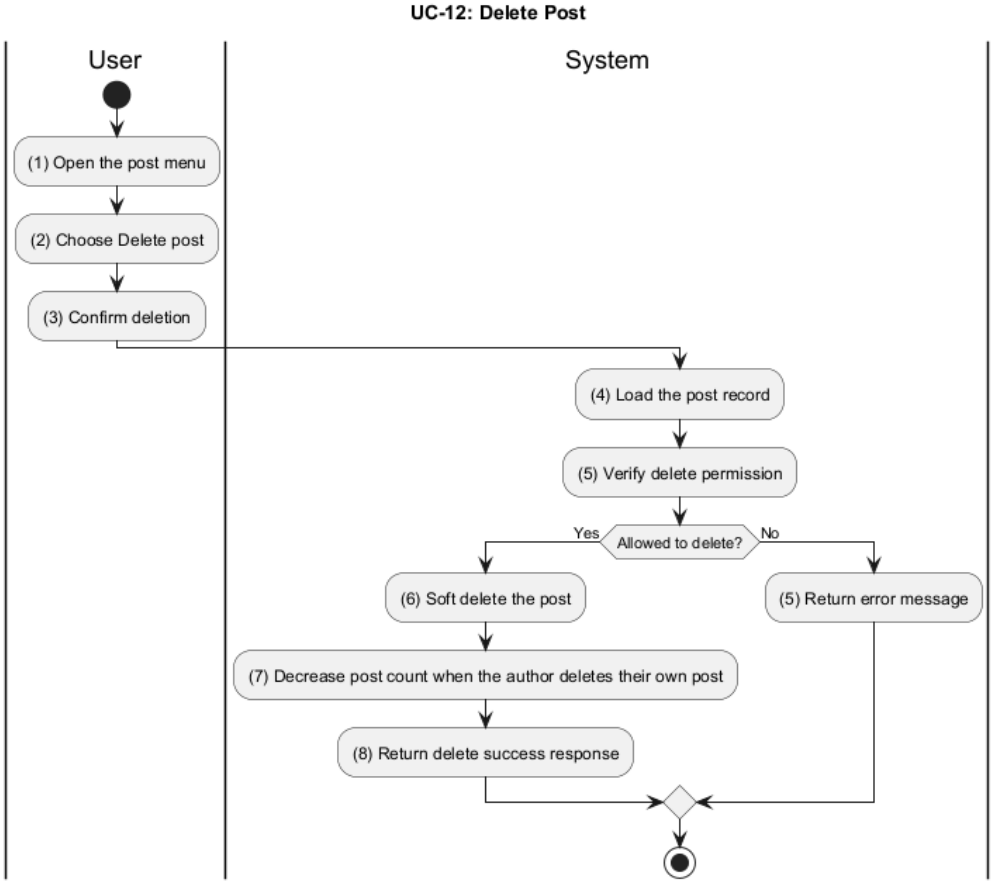


Figure 3.13: Activity Diagram for UC-12

| Activity | BR Code | Description |
|---|---|---|
| (4–5) | BR1 | **Verify Delete Permission:**<br>• The system loads the post record and checks whether the requester is the author.<br>• If the requester is not the author, the system checks for admin, creator, or moderator permission.<br>• If no permission exists, the system returns a forbidden error. |
| (6–8) | BR2 | **Soft Delete the Post:**<br>• The system marks the post as deleted in storage.<br>• If the author deletes their own post, the system decreases the author's post count.<br>• The system returns a delete success response. |

Table 3.23: Business Rules for UC-12 - Delete Post

### 3.1.13 UC-13 - Post Comment

| | |
|---|---|
| **Name** | Post Comment |
| **Description** | This use case describes how a user adds a direct comment to a post. |
| **Actor** | User |
| **Trigger** | When the user submits a comment on a post. |
| **Pre-condition** | The user is logged in to the system.<br>The target post is visible and commentable. |
| **Post-condition** | The comment is created and attached to the post.<br>The comment count is updated. |

Table 3.24: Use Case Specification - Post Comment

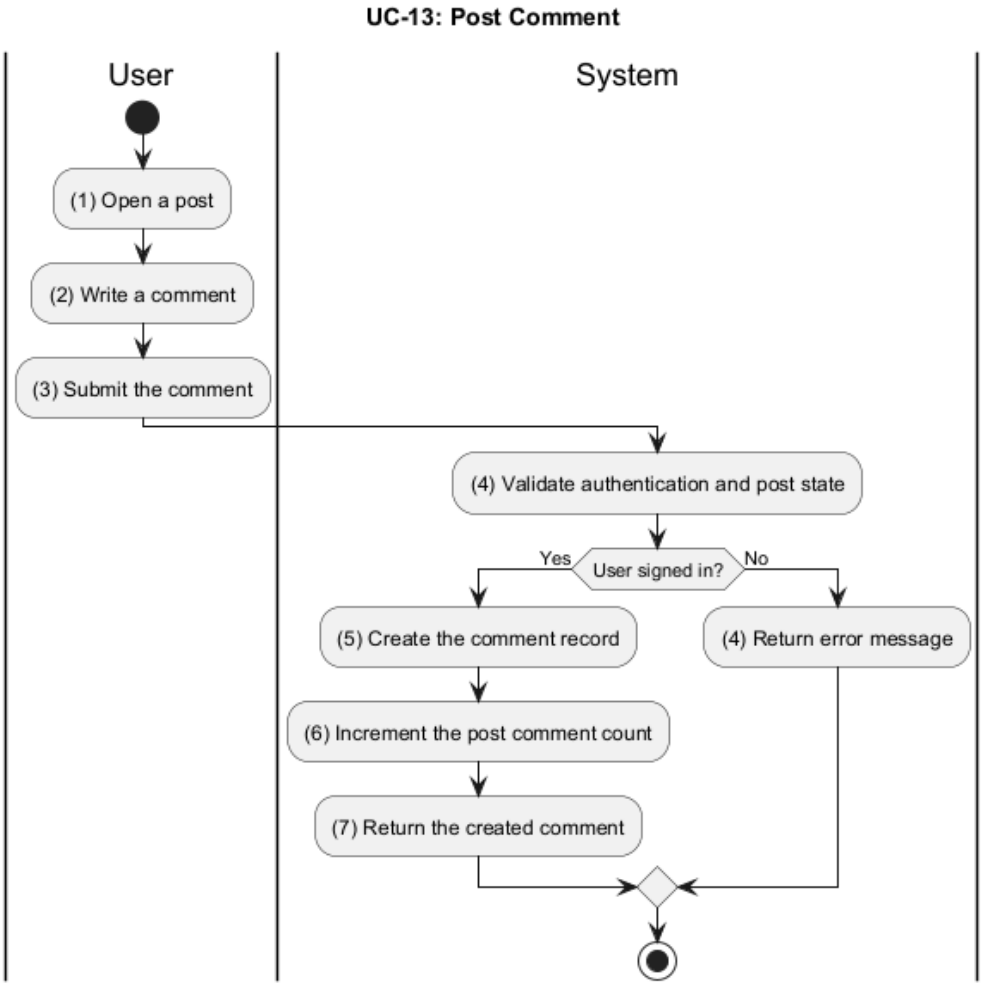


Figure 3.14: Activity Diagram for UC-13

| Activ-ity | BR Code | Description |
|---|---|---|
| (4) | BR1 | **Validate Post and Permissions:**<br>• The system validates authentication and checks that the target post exists.<br>• The system blocks the comment if the user is banned or muted in the post's community. |
| (5–7) | BR2 | **Create the Comment:**<br>• The system stores the new comment with the current user as author.<br>• The system increments the post comment count and the user's comment count.<br>• The system publishes the comment-created event for follow-up processing. |

Table 3.25: Business Rules for UC-13 - Post Comment

### 3.1.14 UC-14 - Reply to Comment

| | |
|---|---|
| **Name** | Reply to Comment |
| **Description** | This use case describes how a user replies to an existing comment in a nested discussion thread. |
| **Actor** | User |
| **Trigger** | When the user submits a reply to a comment. |
| **Pre-condition** | The user is logged in to the system.<br>The parent comment exists and can receive replies. |
| **Post-condition** | The reply is created as a nested comment.<br>The discussion thread is updated. |

Table 3.26: Use Case Specification - Reply to Comment

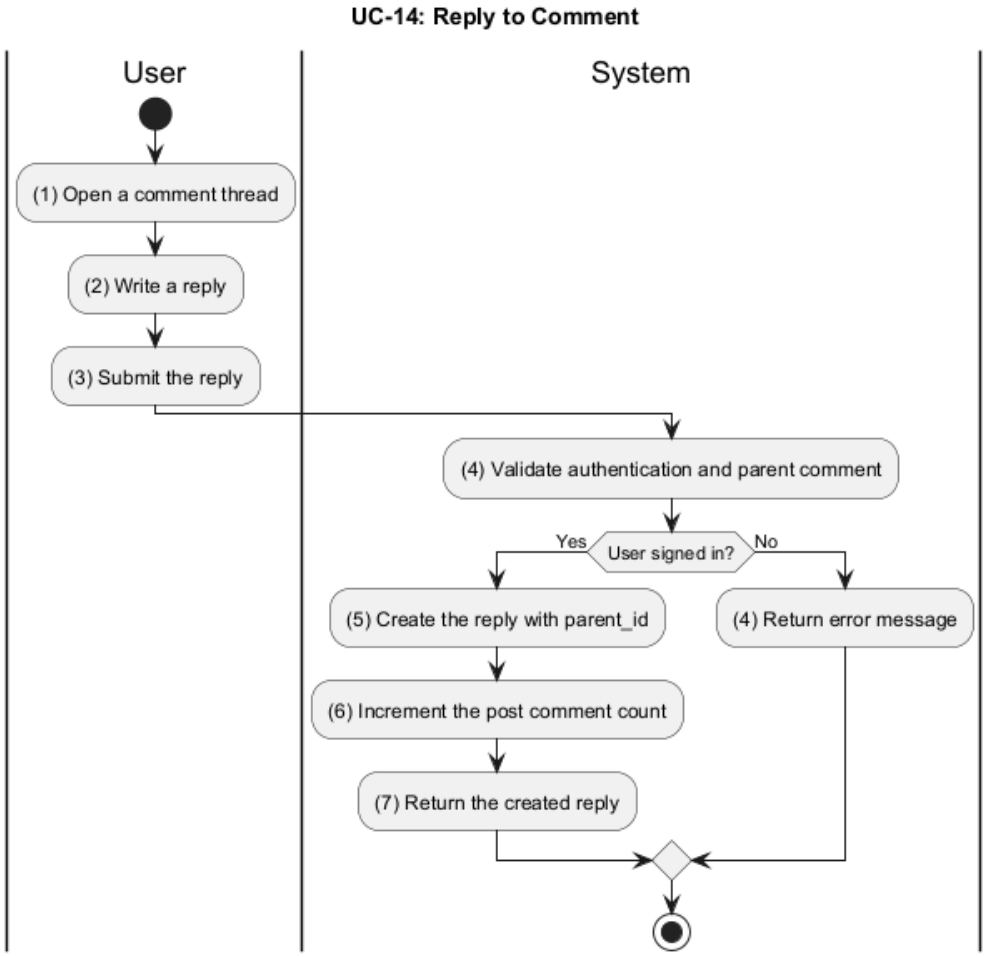


Figure 3.15: Activity Diagram for UC-14

| Activity | BR Code | Description |
|---|---|---|
| (4) | BR1 | **Validate Parent Comment:**<br>• The system validates authentication and checks the parent comment when a reply is submitted.<br>• The system loads the parent comment to derive the parent author for notification logic. |
| (5–7) | BR2 | **Create the Nested Reply:**<br>• The system stores the reply with a populated **parent_id**.<br>• The system increments the post comment count and returns the created reply. |

Table 3.27: Business Rules for UC-14 - Reply to Comment

### 3.1.15 UC-15 - Vote on Content

| | |
|---|---|
| **Name** | Vote on Content |
| **Description** | This use case describes how a user upvotes or downvotes a post or comment. |
| **Actor** | User |
| **Trigger** | When the user selects an upvote or downvote action. |
| **Pre-condition** | The user is logged in to the system.<br>The target content is available for voting. |
| **Post-condition** | The vote is recorded or updated.<br>The content score is recalculated. |

Table 3.28: Use Case Specification - Vote on Content

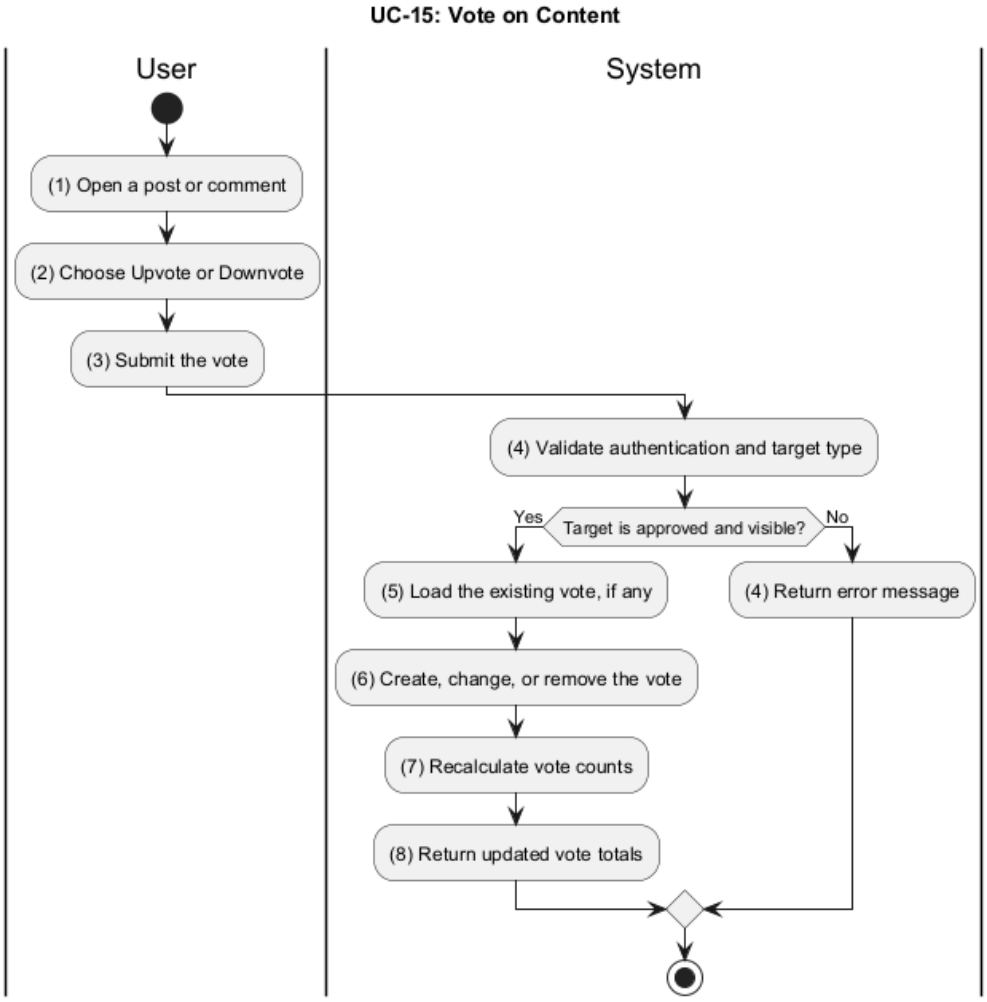


Figure 3.16: Activity Diagram for UC-15

| Activity | BR Code | Description |
|---|---|---|
| (4) | BR1 | **Validate Voting Target:**<br>• The system validates authentication and checks the target type.<br>• The system allows voting only when the post is approved or skipped, or when the target is a visible comment. |
| (5–8) | BR2 | **Apply Vote and Return Counts:**<br>• The system loads the existing vote using ***GetUserVote()***.<br>• The system creates, changes, or removes the vote using ***VoteOnTarget()***.<br>• The system recalculates vote counts and returns the updated totals. |

Table 3.29: Business Rules for UC-15 - Vote on Content

### 3.1.16 UC-16 - Participate in Poll

| | |
|---|---|
| **Name** | Participate in Poll |
| **Description** | This use case describes how a user casts a vote in a community poll. |
| **Actor** | User |
| **Trigger** | When the user submits a poll vote. |
| **Pre-condition** | The user is logged in to the system.<br>The poll is open for voting. |
| **Post-condition** | The selected poll option is recorded.<br>The poll results are updated. |

Table 3.30: Use Case Specification - Participate in Poll

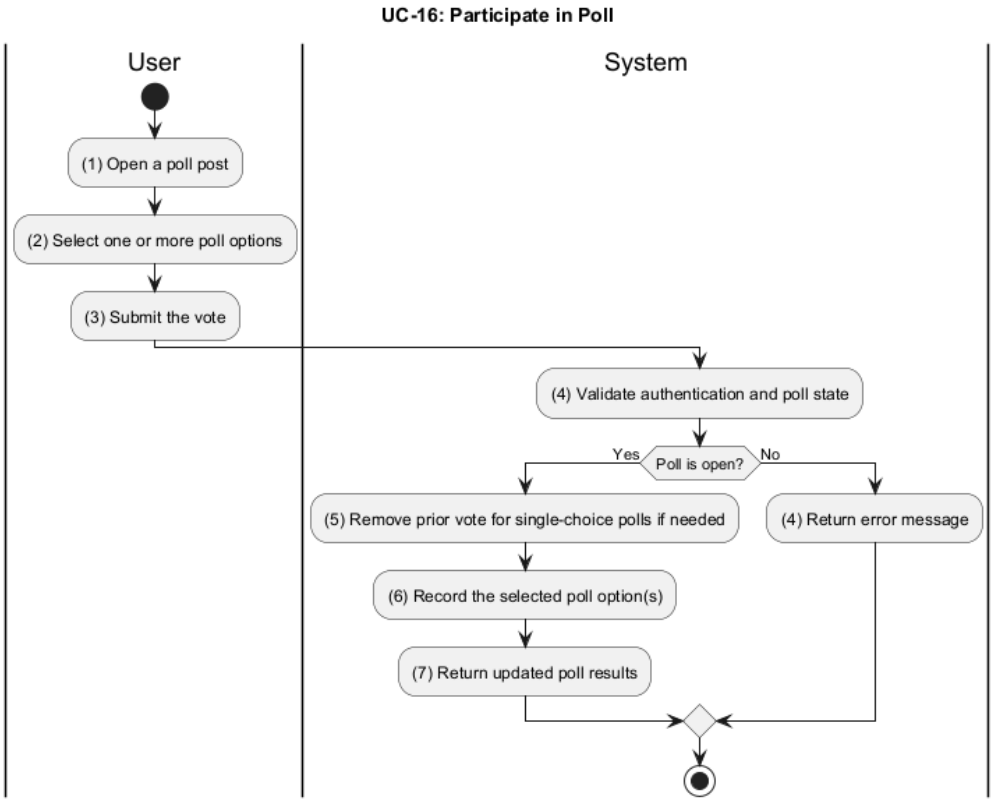


Figure 3.17: Activity Diagram for UC-16

| Activity | BR Code | Description |
|---|---|---|
| (4) | BR1 | **Validate Poll State:**<br>• The system validates authentication and checks whether the poll is open.<br>• The system rejects votes when the poll is closed or unavailable. |
| (5–7) | BR2 | **Record the Poll Vote:**<br>• The system removes a previous vote when the poll is single-choice.<br>• The system records the selected poll option(s) using ***VoteOnPoll()***.<br>• The system returns the updated poll results to the client. |

Table 3.31: Business Rules for UC-16 - Participate in Poll

### 3.1.17 UC-17 - Save Post

| | |
|---|---|
| **Name** | Save Post |
| **Description** | This use case describes how a user bookmarks a post for later viewing. |
| **Actor** | User |
| **Trigger** | When the user clicks the “Save” action on a post. |
| **Pre-condition** | The user is logged in to the system.<br>The target post exists and is visible to the user. |
| **Post-condition** | The post is added to the user’s saved list.<br>The saved state is available in the account. |

Table 3.32: Use Case Specification - Save Post

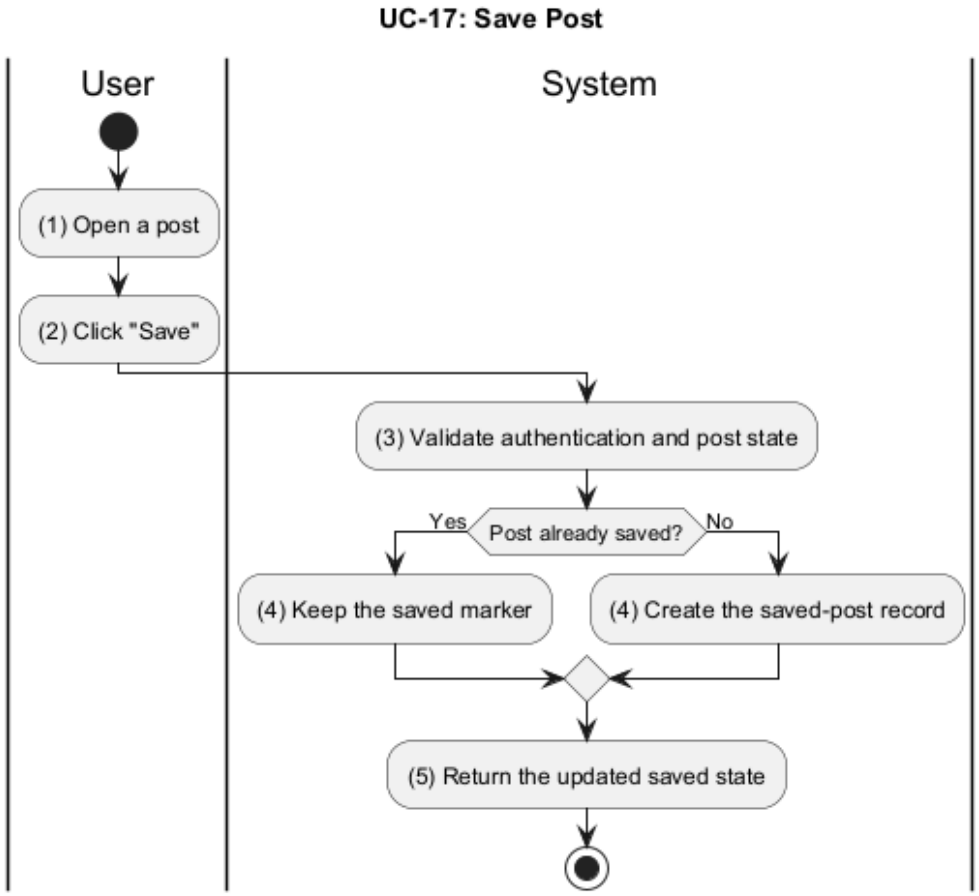


Figure 3.18: Activity Diagram for UC-17

| Activity | BR Code | Description |
|---|---|---|
| (3–5) | BR1 | **Save or Unsave the Post:**<br>• The system validates authentication and checks that the post exists.<br>• The system stores the saved-post relation using ***SavePost()***.<br>• If the post is already saved, the system keeps the saved marker instead of duplicating it. |

Table 3.33: Business Rules for UC-17 - Save Post

### 3.1.18 UC-18 - Hide Post

| | |
|---|---|
| **Name** | Hide Post |
| **Description** | This use case describes how a user hides a post from their personal feed. |
| **Actor** | User |
| **Trigger** | When the user clicks the “Hide” action on a post. |
| **Pre-condition** | The user is logged in to the system.<br>The target post is visible in the user’s feed. |
| **Post-condition** | The post is hidden from the user’s feed.<br>The hidden-post record is stored for the account. |

Table 3.34: Use Case Specification - Hide Post

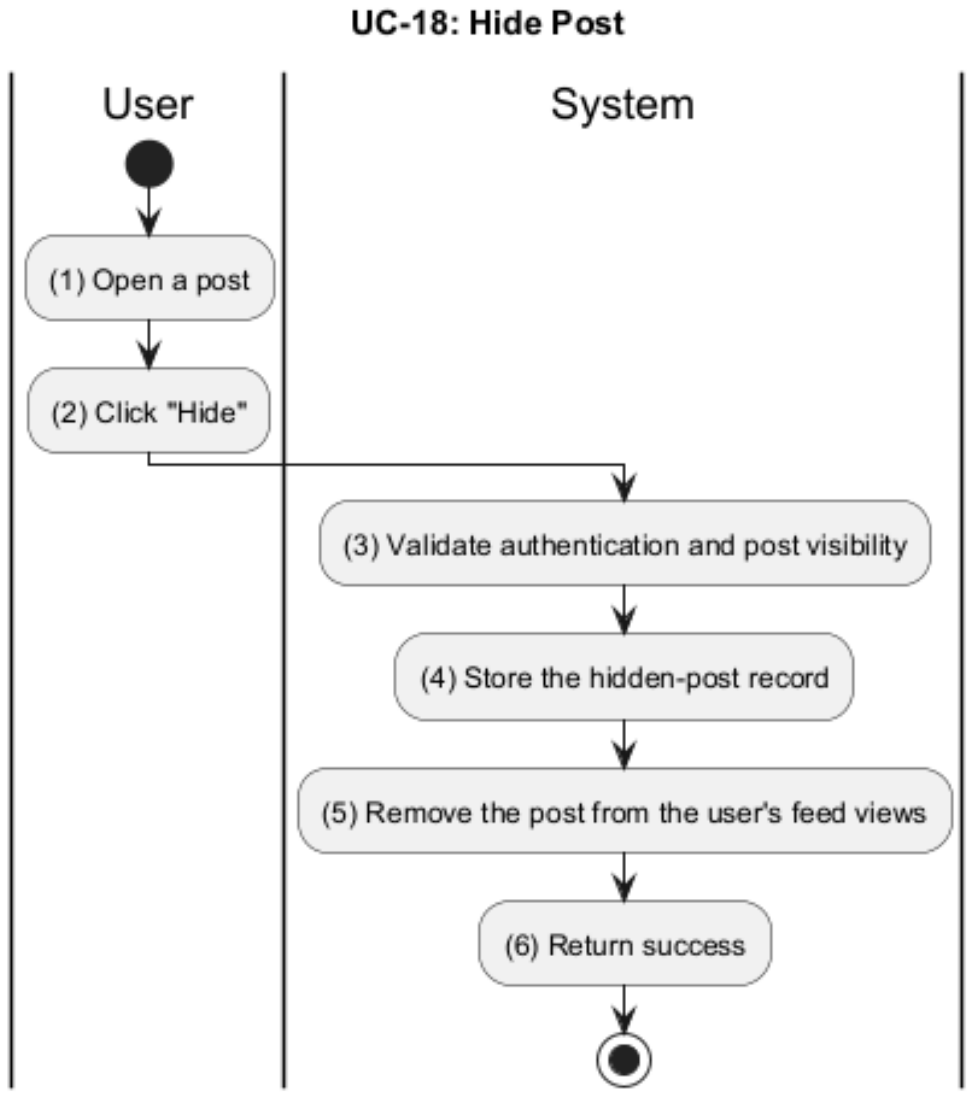


Figure 3.19: Activity Diagram for UC-18

| Activity | BR Code | Description |
|---|---|---|
| (3–6) | BR1 | **Hide the Post:**<br>• The system validates authentication and confirms that the requester is the post author.<br>• The system marks the post as hidden using ***HidePost()***.<br>• The system removes the post from the user's visible feed views. |

Table 3.35: Business Rules for UC-18 - Hide Post

### 3.1.19 UC-19 - Manage Drafts

| | |
|---|---|
| **Name** | Manage Drafts |
| **Description** | This use case describes how a user saves unfinished posts as drafts and publishes them later. |
| **Actor** | User |
| **Trigger** | When the user saves, edits, opens, or publishes a draft. |
| **Pre-condition** | The user is logged in to the system.<br>The user has draft access. |
| **Post-condition** | The draft is saved, updated, or deleted after publishing.<br>A published draft becomes a normal post. |

Table 3.36: Use Case Specification - Manage Drafts

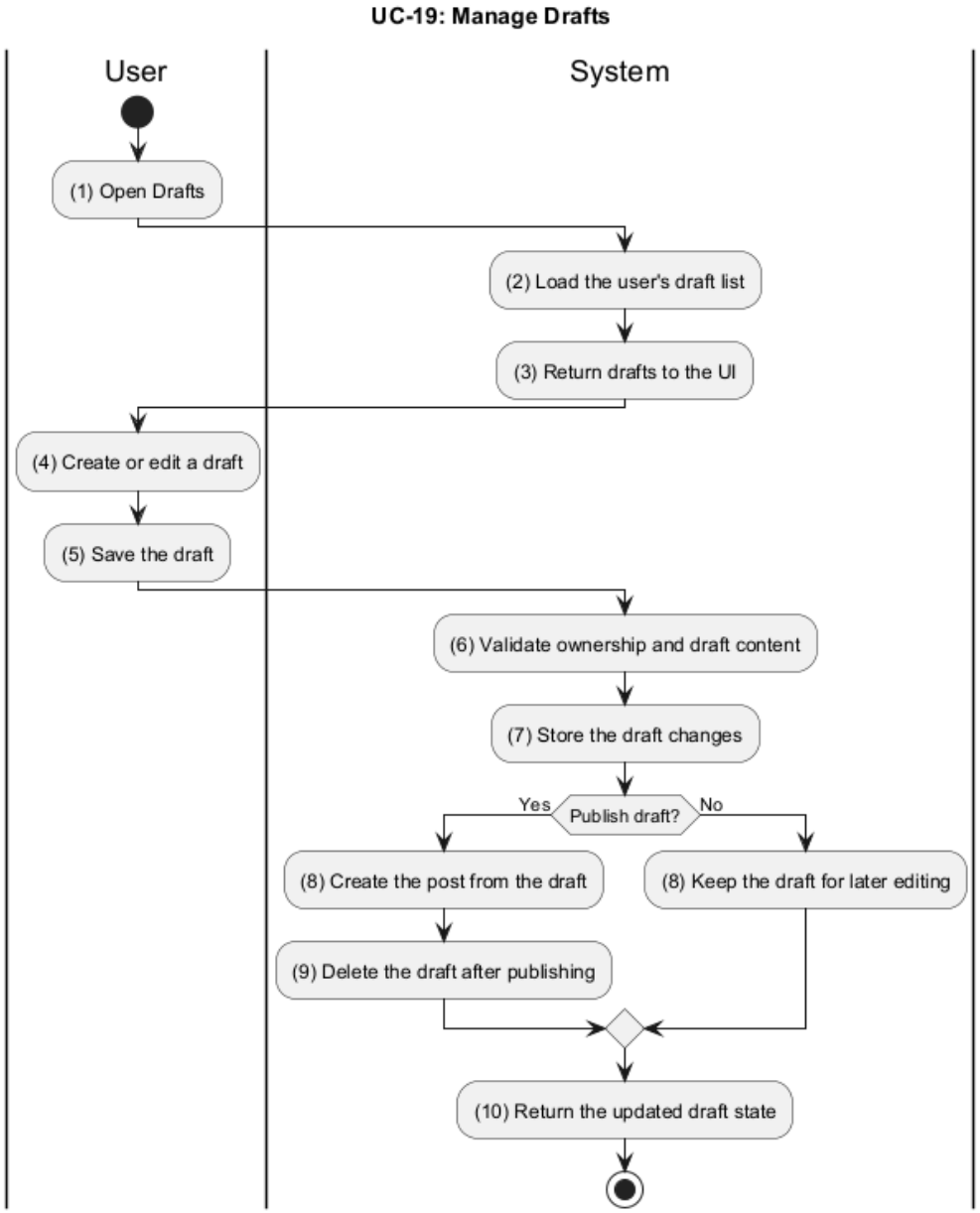


Figure 3.20: Activity Diagram for UC-19

| **Activity** | **BR Code** | **Description** |
|---|---|---|
| (2–3) | BR1 | **Load Drafts:**<br>• The system loads the authenticated user's draft list using ***GetDraftsByAuthor()***.<br>• The system returns drafts together with pagination data. |

Table 3.37: Business Rules for UC-19 - Manage Drafts

| Activity | BR Code | Description |
|---|---|---|
| (6–7) | BR2 | **Create or Update Draft:**<br>• The system validates draft ownership before saving changes.<br>• The system stores the draft via ***CreateDraft()*** or ***UpdateDraft()***.<br>• The system updates text, images, videos, poll data, tags, and community selection on the draft record. |
| (8–10) | BR3 | **Publish or Delete Draft:**<br>• The system checks required fields before publishing a draft.<br>• The system creates the post from the draft using ***PublishDraft()***.<br>• After successful publishing, the system deletes the draft record. |

Table 3.37: Business Rules for UC-19 - Manage Drafts

### 3.1.20 UC-20 - Private Messaging

| | |
|---|---|
| **Name** | Private Messaging |
| **Description** | This use case describes how users exchange instant one-to-one messages through the messaging system. |
| **Actor** | User |
| **Trigger** | When the user opens a conversation and sends a message. |
| **Pre-condition** | The user is logged in to the system.<br>The conversation channel exists and is accessible. |
| **Post-condition** | The message is delivered to the conversation members.<br>The channel read state is updated. |

Table 3.38: Use Case Specification - Private Messaging

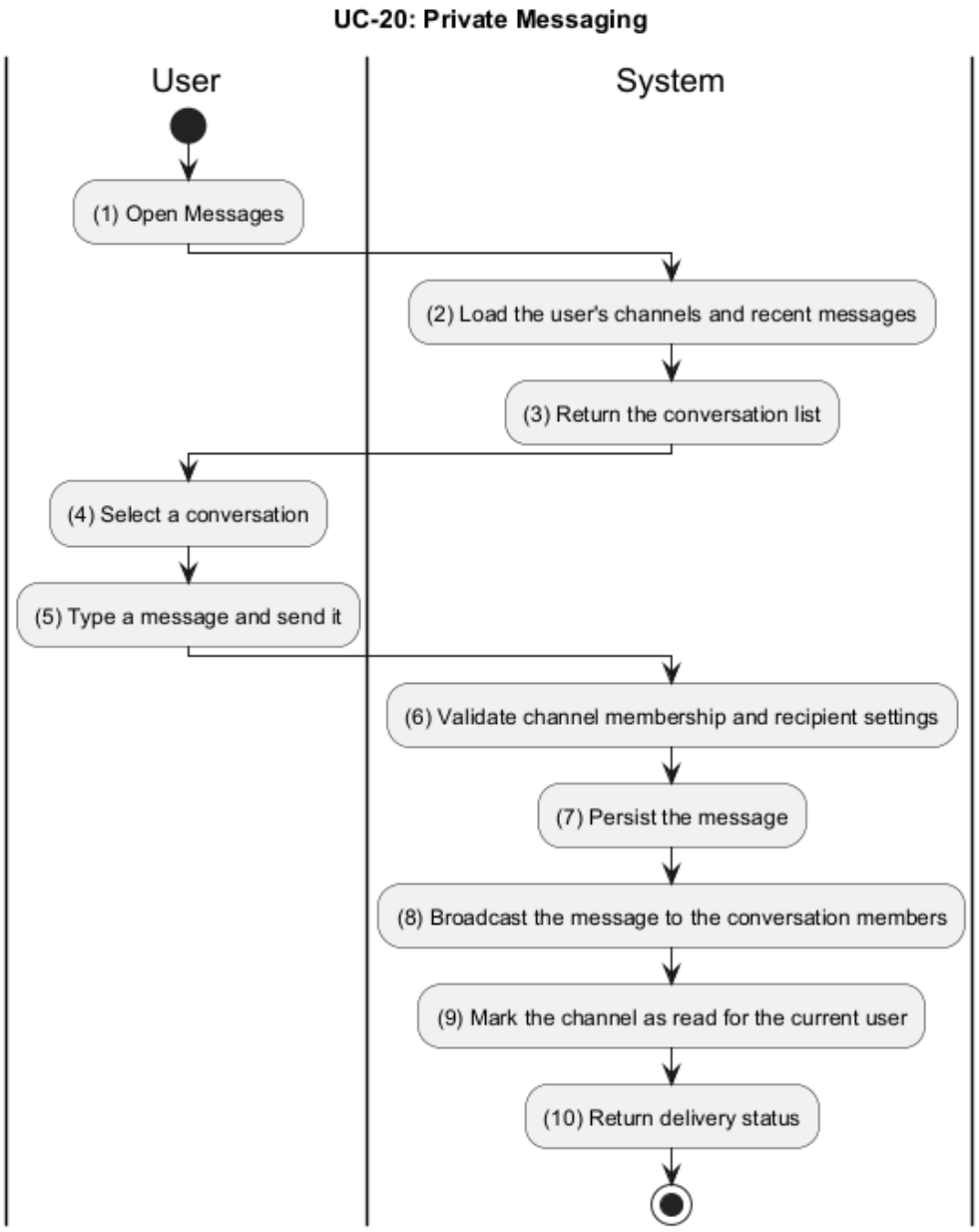


Figure 3.21: Activity Diagram for UC-20

| Activity | BR Code | Description |
|---|---|---|
| (2–3) | BR1 | **Load Conversations:**<br>• The system loads the authenticated user's channels using ***GetChannelsByUserID()***.<br>• The system returns the conversation list and recent message state. |
| (4–10) | BR2 | **Send and Deliver Message:**<br>• The system validates that the sender is a channel member and that the recipients allow direct messages.<br>• The system persists the message and broadcasts it to the other channel members.<br>• The system marks the channel as read for the active user using ***MarkChannelAsRead()***. |

Table 3.39: Business Rules for UC-20 - Private Messaging

### 3.1.21 UC-21 - Manage Notifications

| | |
|---|---|
| **Name** | Manage Notifications |
| **Description** | This use case describes how a user views and clears system notifications. |
| **Actor** | User |
| **Trigger** | When the user opens the notifications page or marks notifications as read. |
| **Pre-condition** | The user is logged in to the system.<br>Notifications exist for the account. |
| **Post-condition** | The selected notifications are marked as read or deleted.<br>The notification list is refreshed. |

Table 3.40: Use Case Specification - Manage Notifications

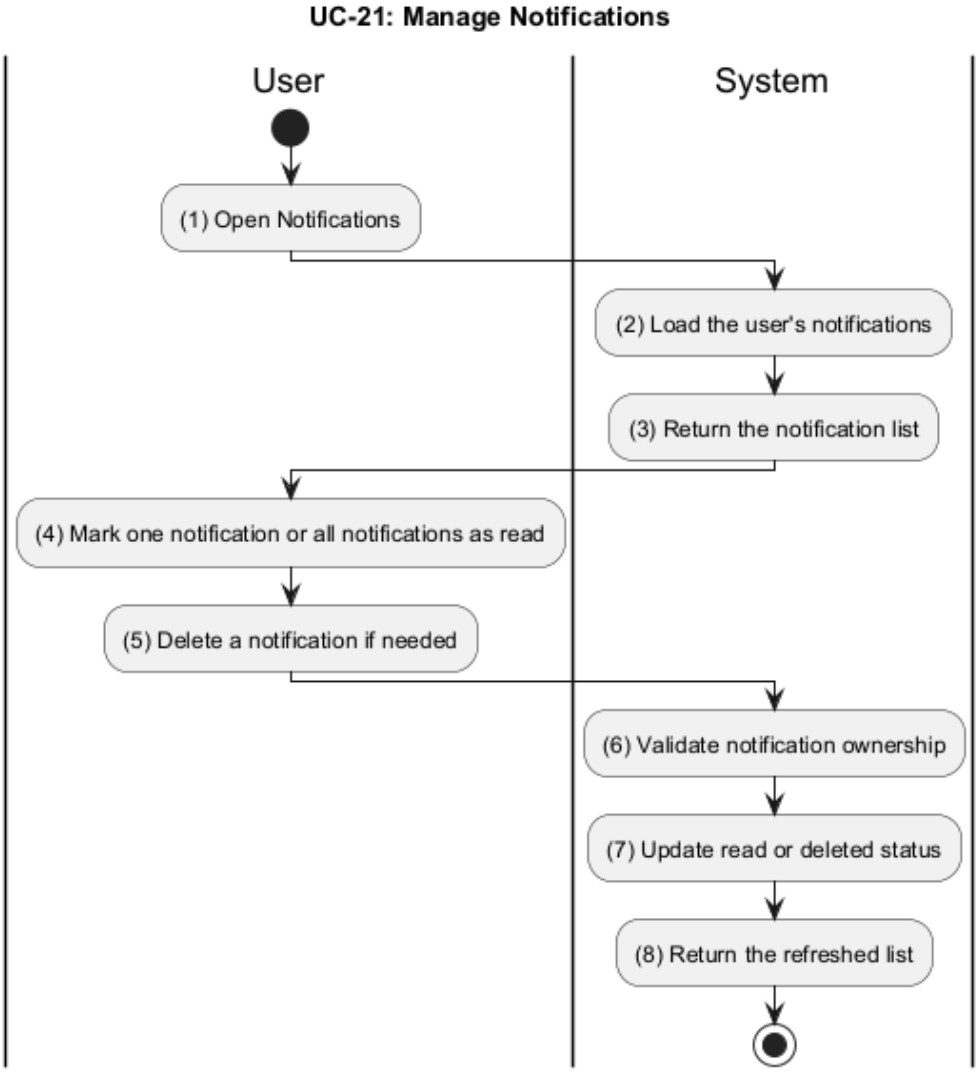


Figure 3.22: Activity Diagram for UC-21

| Activity | BR Code | Description |
|---|---|---|
| (2–3) | BR1 | **Load Notifications:**<br>• The system loads notifications for **recipientID** using ***GetNotifications()***.<br>• The system returns the notification list with pagination data. |

Table 3.41: Business Rules for UC-21 - Manage Notifications

| Activity | BR Code | Description |
|---|---|---|
| (4–8) | BR2 | **Update Notification State:**<br>• The system validates notification ownership before changing any record.<br>• If the user marks all notifications as read, the system updates them using ***MarkAllAsRead()***.<br>• If the user marks a single notification as read, the system validates **notificationID**, converts it to an object ID, and updates it using ***MarkAsRead()***.<br>• If the user deletes a notification, the system validates **notificationID** and removes it using ***DeleteNotification()***.<br>• The system returns the refreshed notification list after the update. |

Table 3.41: Business Rules for UC-21 - Manage Notifications

### 3.1.22 UC-22 - Report Violation

| | |
|---|---|
| **Name** | Report Violation |
| **Description** | This use case describes how a user reports inappropriate content or behavior for review. |
| **Actor** | User |
| **Trigger** | When the user submits a report form. |
| **Pre-condition** | The user is logged in to the system.<br>The reported target is available. |
| **Post-condition** | The report is stored in the moderation queue.<br>The report can be reviewed by moderators or admins. |

Table 3.42: Use Case Specification - Report Violation

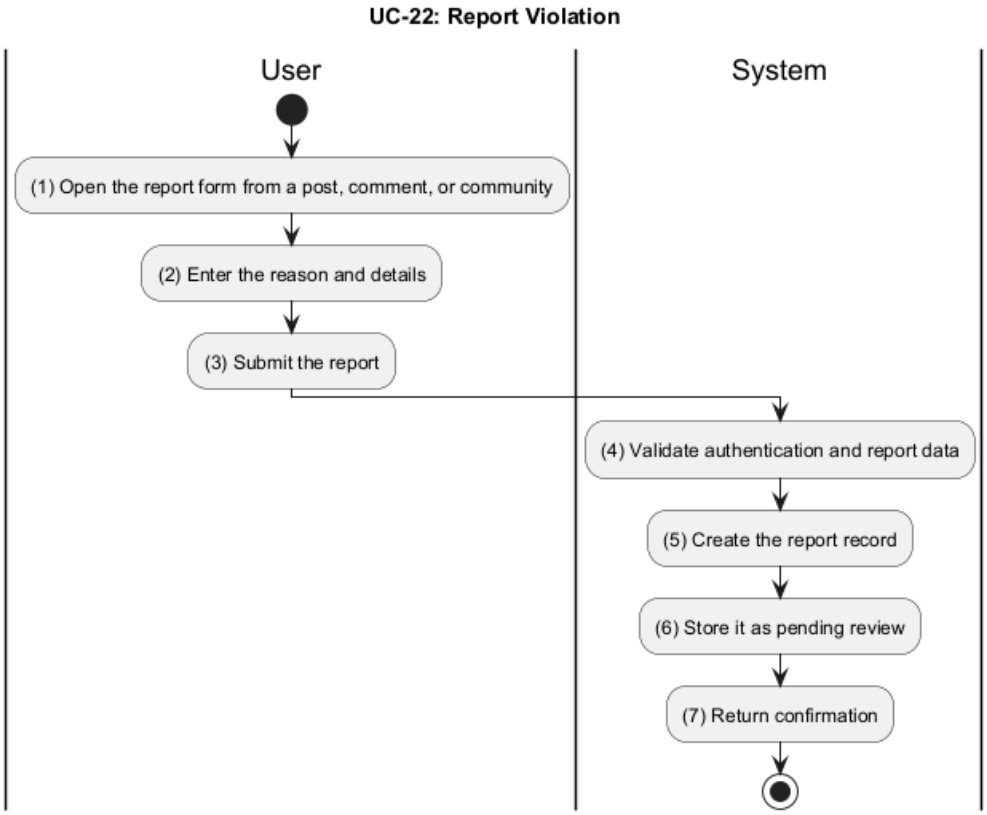


Figure 3.23: Activity Diagram for UC-22

| **Activity** | **BR Code** | **Description** |
|---|---|---|
| (1–3) | BR1 | **Validate Report Submission:**<br>• The system checks that the user is authenticated.<br>• The system validates the submitted **reason** and **details** in ***ReportPost()***.<br>• The system checks that the target post exists and that **reporterID** is not the post author. |
| (4–7) | BR2 | **Create Report Record:**<br>• The system creates a new report with **Create()** on the report repository.<br>• The system stores the report as pending review.<br>• The system returns a confirmation message after the report is saved. |

Table 3.43: Business Rules for UC-22 - Report Violation

### 3.1.23 UC-23 - Create Community

| | |
|---|---|
| **Name** | Create Community |
| **Description** | This use case describes how a user creates a new community and becomes its owner. |
| **Actor** | User |
| **Trigger** | When the user submits the create community form. |
| **Pre-condition** | The user is logged in to the system.<br>The community name is available and valid. |
| **Post-condition** | The community is created successfully.<br>The creator becomes the owner and first member. |

Table 3.44: Use Case Specification - Create Community

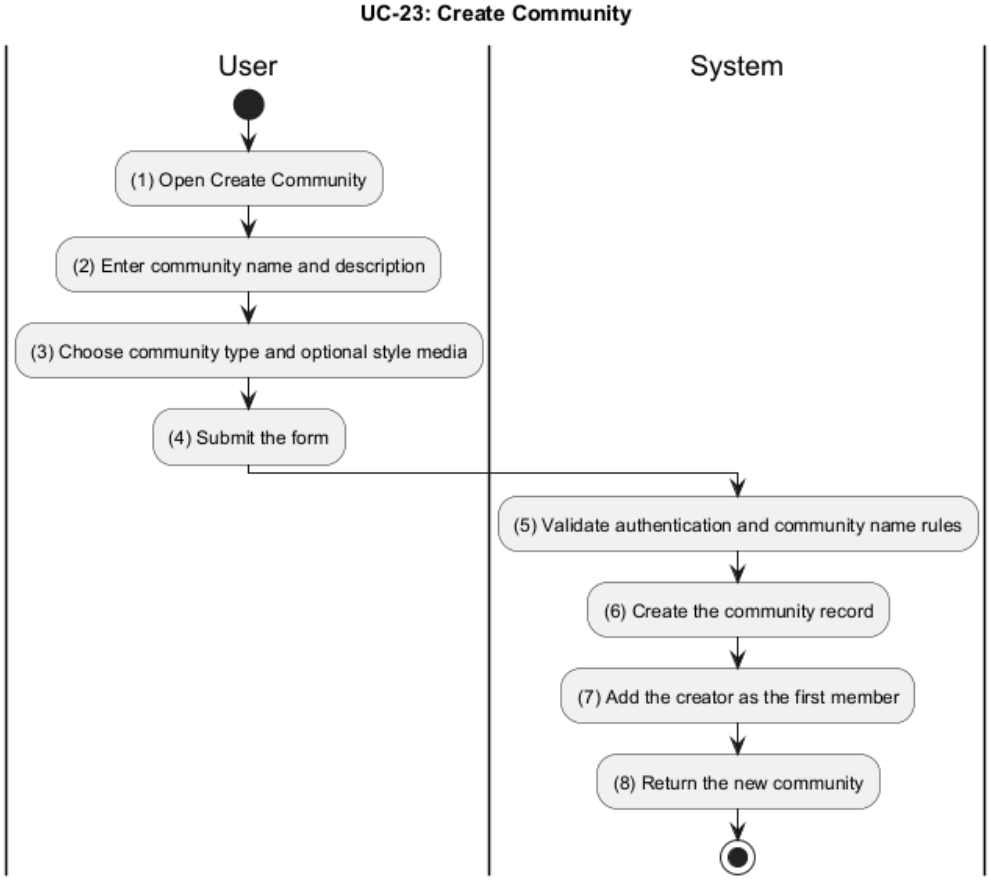


Figure 3.24: Activity Diagram for UC-23

| Activity | BR Code | Description |
|---|---|---|
| (1–4) | BR1 | **Validate Community Input:**<br>• The system checks that the user is authenticated.<br>• The system validates the submitted community fields, especially **name** in ***CreateCommunity()***.<br>• The system rejects duplicate community names with a community-name-exists error. |
| (5–8) | BR2 | **Create Community and Initial Membership:**<br>• The system creates the community record with ***CreateCommunity()***.<br>• The system adds the creator as the first member by creating a membership record.<br>• The system returns the new community after persistence succeeds. |

Table 3.45: Business Rules for UC-23 - Create Community

### 3.1.24 UC-24 - Approve / Reject Post

| | |
|---|---|
| **Name** | Approve / Reject Post |
| **Description** | This use case describes how a moderator reviews pending or edited posts and decides whether to publish them. |
| **Actor** | Moderator |
| **Trigger** | When the moderator opens the moderation queue and chooses approve or reject. |
| **Pre-condition** | The moderator is logged in to the system.<br>The target post is pending review or edited. |
| **Post-condition** | The post is approved or rejected.<br>The moderation result is stored and reflected in the community. |

Table 3.46: Use Case Specification - Approve / Reject Post

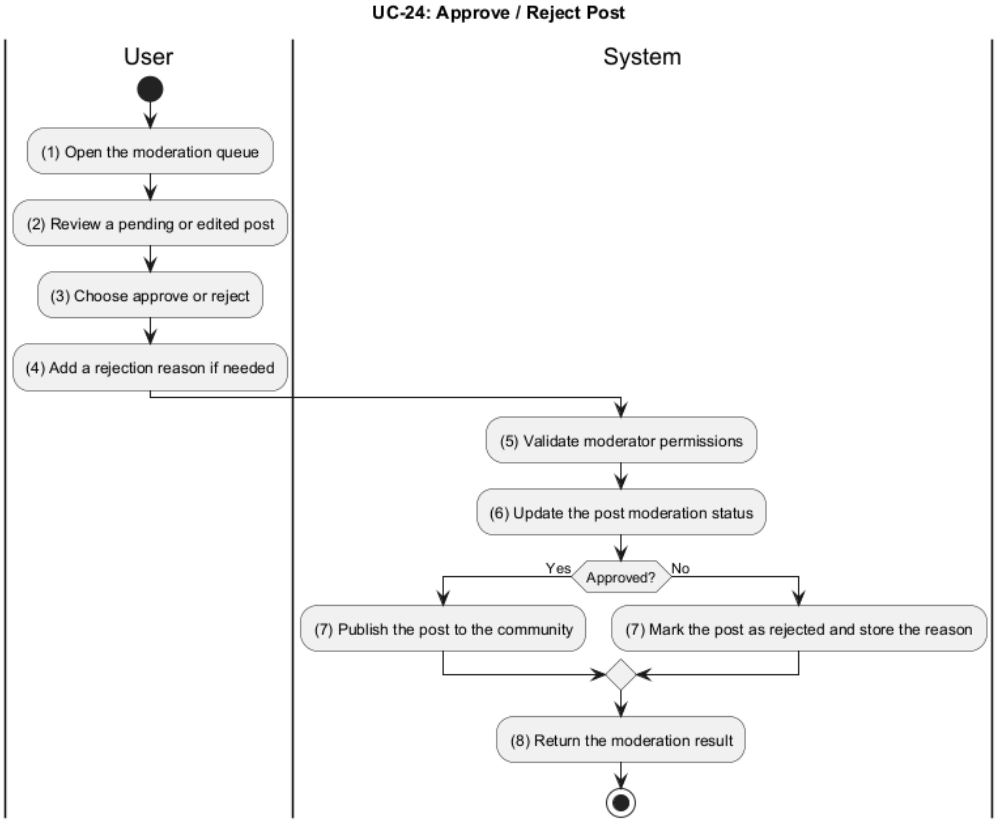


Figure 3.25: Activity Diagram for UC-24

| **Activity** | **BR Code** | **Description** |
|---|---|---|
| (1–4) | BR1 | **Prepare Moderation Decision:**<br>• The system loads the moderation queue using ***GetPendingPosts()*** and ***GetEditedPosts()***.<br>• The system validates that the requester has moderator access.<br>• The system checks that the selected post belongs to the current community and is still pending moderation. |

Table 3.47: Business Rules for UC-24 - Approve / Reject Post

| Activity | BR Code | Description |
|---|---|---|
| (5–8) | BR2 | **Apply Moderation Result:**<br>• The system updates the post using ***ModeratePost()***.<br>• If the moderator approves the post, the system sets the moderation status to approved and publishes the post to the community.<br>• If the moderator rejects the post, the system sets the moderation status to rejected and stores the rejection **reason**.<br>• The system returns the moderation result to the caller. |

Table 3.47: Business Rules for UC-24 - Approve / Reject Post

### 3.1.25 UC-25 - Ban / Mute User

| | |
|---|---|
| **Name** | Ban / Mute User |
| **Description** | This use case describes how a moderator restricts a user's participation in a community. |
| **Actor** | Moderator |
| **Trigger** | When the moderator chooses to ban or mute a user. |
| **Pre-condition** | The moderator is logged in to the system.<br>The target user belongs to the community. |
| **Post-condition** | The ban or mute restriction is stored.<br>The user is prevented from performing the restricted actions. |

Table 3.48: Use Case Specification - Ban / Mute User

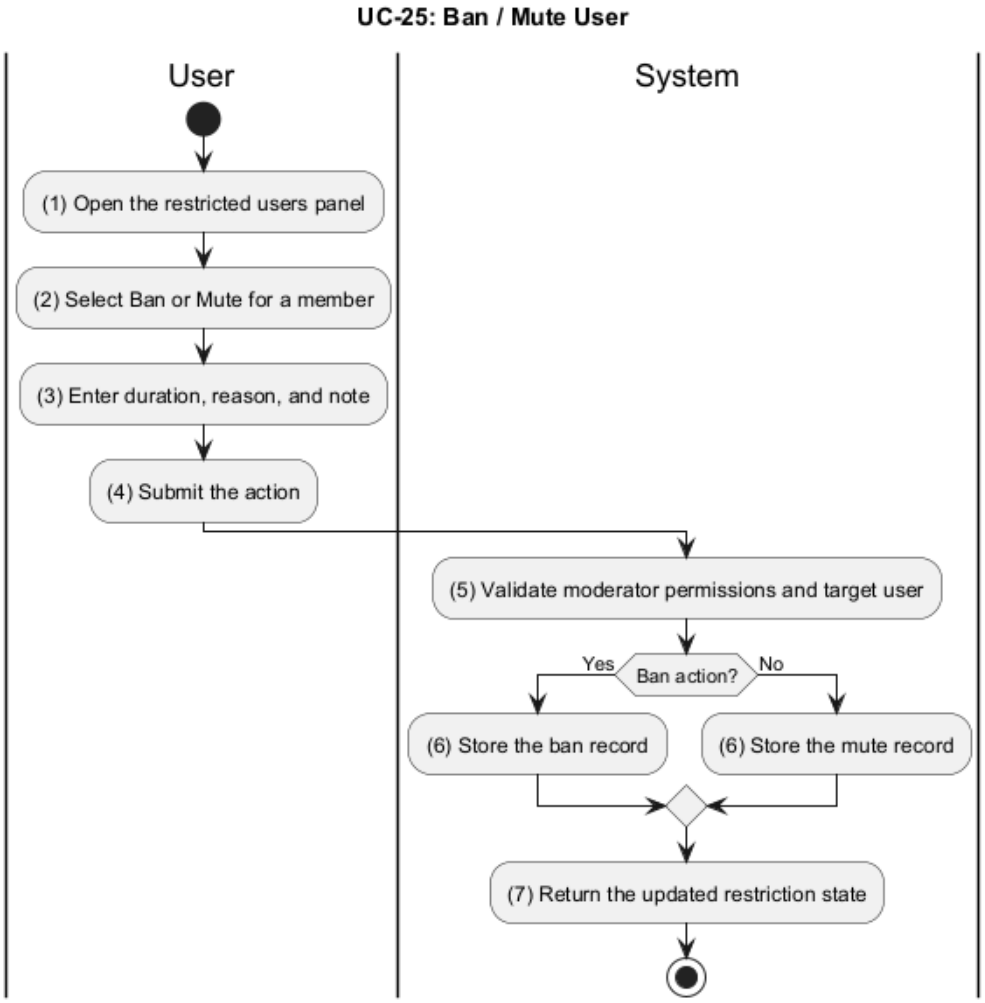


Figure 3.26: Activity Diagram for UC-25

| **Activity** | **BR Code** | **Description** |
|---|---|---|
| (1–4) | BR1 | **Validate Restriction Request:**<br>• The system checks that the requester has moderator access.<br>• The system validates the target **userID**, **communityID**, **Type**, **Reason**, and **LengthDays**.<br>• The system accepts only the **banned** or **muted** restriction type. |

Table 3.49: Business Rules for UC-25 - Ban / Mute User

| Activity | BR Code | Description |
|---|---|---|
| (5–7) | BR2 | **Store Ban or Mute Record:**<br>• The system calculates the expiration timestamp from **LengthDays**.<br>• The system creates a community ban record through ***BanUser()***.<br>• The system stores either a ban or mute entry based on **Type** and returns the updated restriction state. |

Table 3.49: Business Rules for UC-25 - Ban / Mute User

### 3.1.26 UC-26 - Configure Community

| | |
|---|---|
| **Name** | Configure Community |
| **Description** | This use case describes how a community owner updates rules, privacy, and appearance settings. |
| **Actor** | Owner |
| **Trigger** | When the owner opens the community settings page and saves changes. |
| **Pre-condition** | The owner is logged in to the system.<br>The owner has permission to manage the community. |
| **Post-condition** | The community configuration is updated.<br>The new settings are applied to the community. |

Table 3.50: Use Case Specification - Configure Community

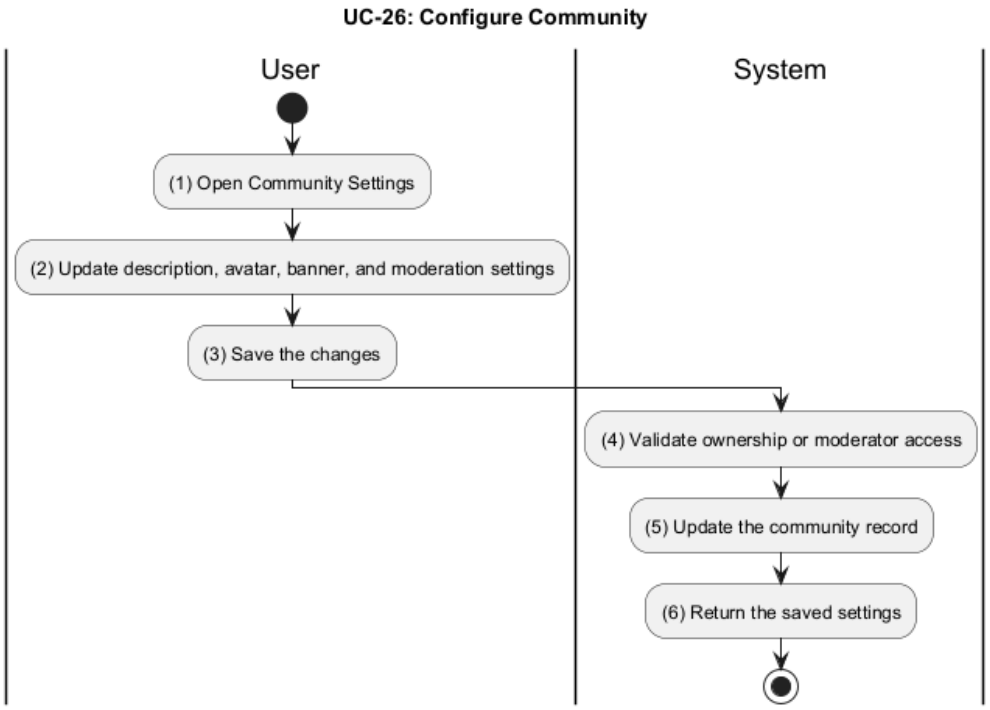


Figure 3.27: Activity Diagram for UC-26

| Activity | BR Code | Description |
|---|---|---|
| (1–3) | BR1 | **Validate Community Access:**<br>• The system checks that the requester has moderator access for the community.<br>• The system validates the incoming community settings before saving them. |
| (4–6) | BR2 | **Persist Community Settings:**<br>• The system updates the community record using ***UpdateCommunity()***.<br>• The system saves the description, avatar, banner, and moderation settings.<br>• The system returns the saved community settings. |

Table 3.51: Business Rules for UC-26 - Configure Community

### 3.1.27 UC-27 - Assign Roles

| | |
|---|---|
| **Name** | Assign Roles |
| **Description** | This use case describes how a community owner appoints moderators or other roles for members. |
| **Actor** | Owner |
| **Trigger** | When the owner invites or activates a role for another member. |
| **Pre-condition** | The owner is logged in to the system.<br>The target member belongs to the community. |
| **Post-condition** | The role assignment is stored successfully.<br>The target member gains the new permissions. |

Table 3.52: Use Case Specification - Assign Roles

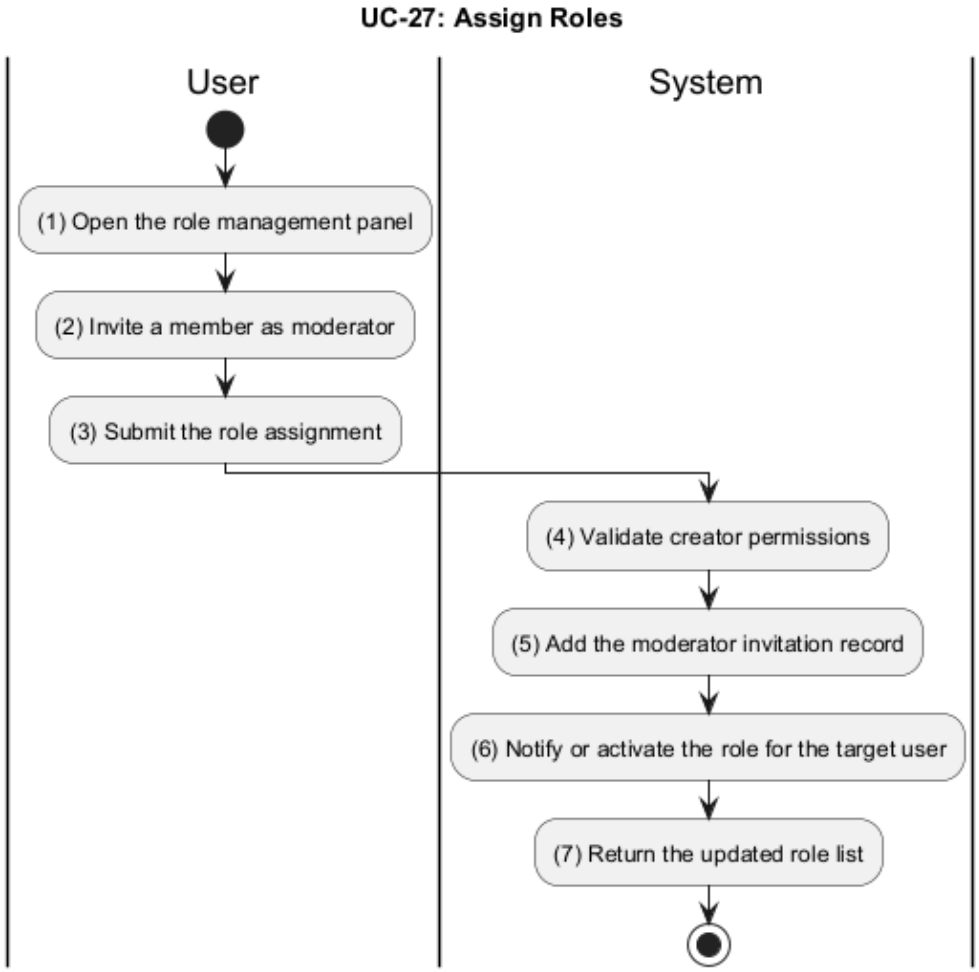


Figure 3.28: Activity Diagram for UC-27

| Activity | BR Code | Description |
|---|---|---|
| (1–3) | BR1 | **Validate Role Assignment:**<br>• The system checks that the requester is the community creator.<br>• The system validates the moderator assignment payload, including **CommunityID** and **AddedModerator**. |
| (4–7) | BR2 | **Add and Activate Moderator Role:**<br>• The system creates the moderator invitation or assignment record using ***AddModerator()***.<br>• The system activates the moderator role for the target user using ***ActivateModerator()*** when the role becomes active.<br>• The system returns the updated moderator list. |

Table 3.53: Business Rules for UC-27 - Assign Roles

### 3.1.28 UC-28 - Handle Global Reports

| | |
|---|---|
| **Name** | Handle Global Reports |
| **Description** | This use case describes how a system admin reviews and acts on platform-wide violation reports. |
| **Actor** | System Admin |
| **Trigger** | When the admin opens the global reports dashboard. |
| **Pre-condition** | The admin is logged in to the system.<br>There are reports available for review. |
| **Post-condition** | The selected reports are reviewed or deleted.<br>The report queue is updated. |

Table 3.54: Use Case Specification - Handle Global Reports

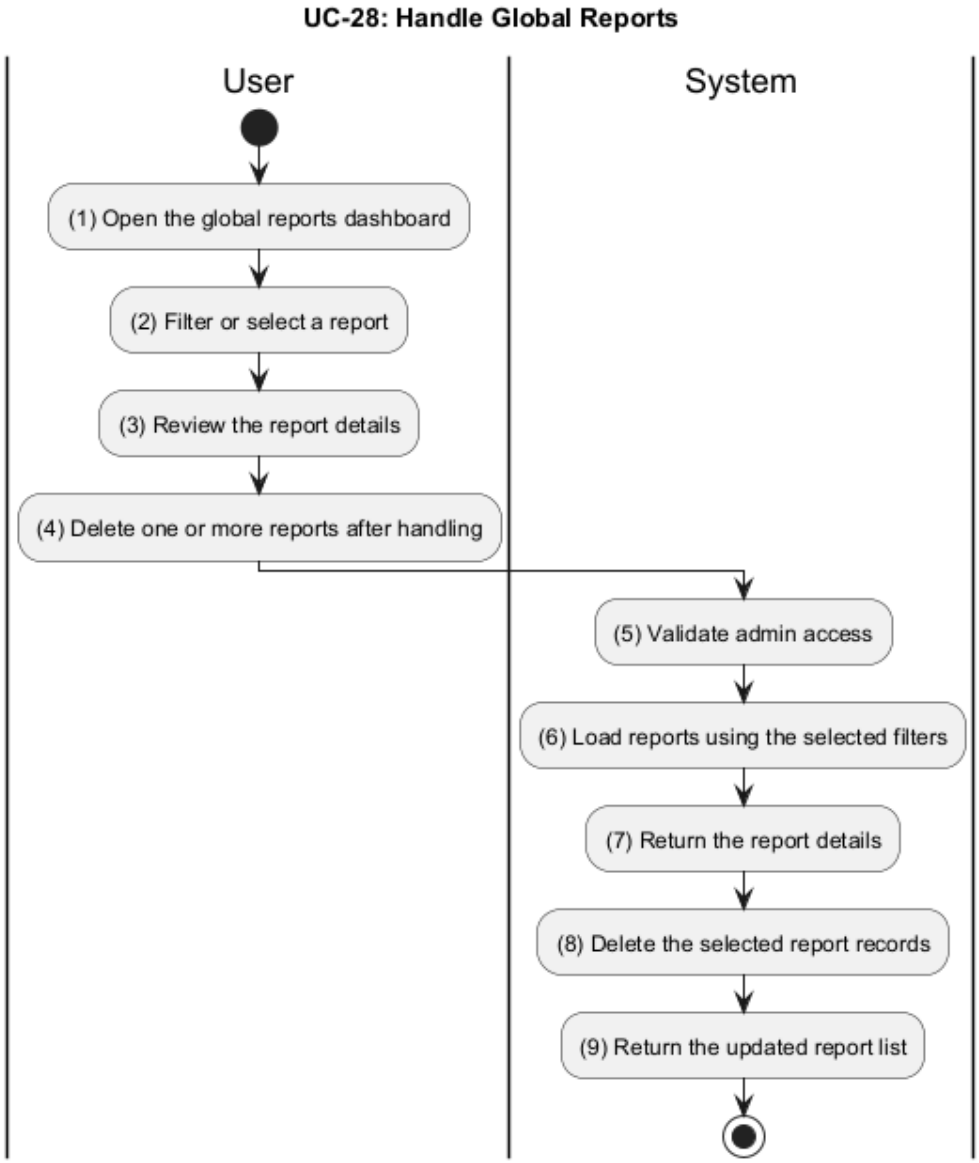


Figure 3.29: Activity Diagram for UC-28

| **Activity** | **BR Code** | **Description** |
|---|---|---|
| (1–3) | BR1 | **Load and Inspect Reports:**<br>• The system checks that the requester is an admin.<br>• The system loads filtered reports using ***GetReportsFilter()***.<br>• The system allows the admin to inspect a single report using ***GetReportByID()***. |

Table 3.55: Business Rules for UC-28 - Handle Global Reports

| Activity | BR Code | Description |
|---|---|---|
| (4–9) | BR2 | **Delete Handled Reports:**<br>• The system deletes a single report using ***DeleteReport()*** or multiple reports using ***DeleteReports()***.<br>• The system validates that the admin is allowed to delete the selected report records.<br>• The system rejects an empty batch request.<br>• The system returns the updated report list after deletion. |

Table 3.55: Business Rules for UC-28 - Handle Global Reports

### 3.1.29 UC-29 - Manage Platform

| | |
|---|---|
| **Name** | Manage Platform |
| **Description** | This use case describes how a system admin manages users and communities globally. |
| **Actor** | System Admin |
| **Trigger** | When the admin opens the platform management area and selects a target entity. |
| **Pre-condition** | The admin is logged in to the system.<br>The admin has global management privileges. |
| **Post-condition** | The selected user or community is banned or unbanned.<br>The platform state is updated accordingly. |

Table 3.56: Use Case Specification - Manage Platform

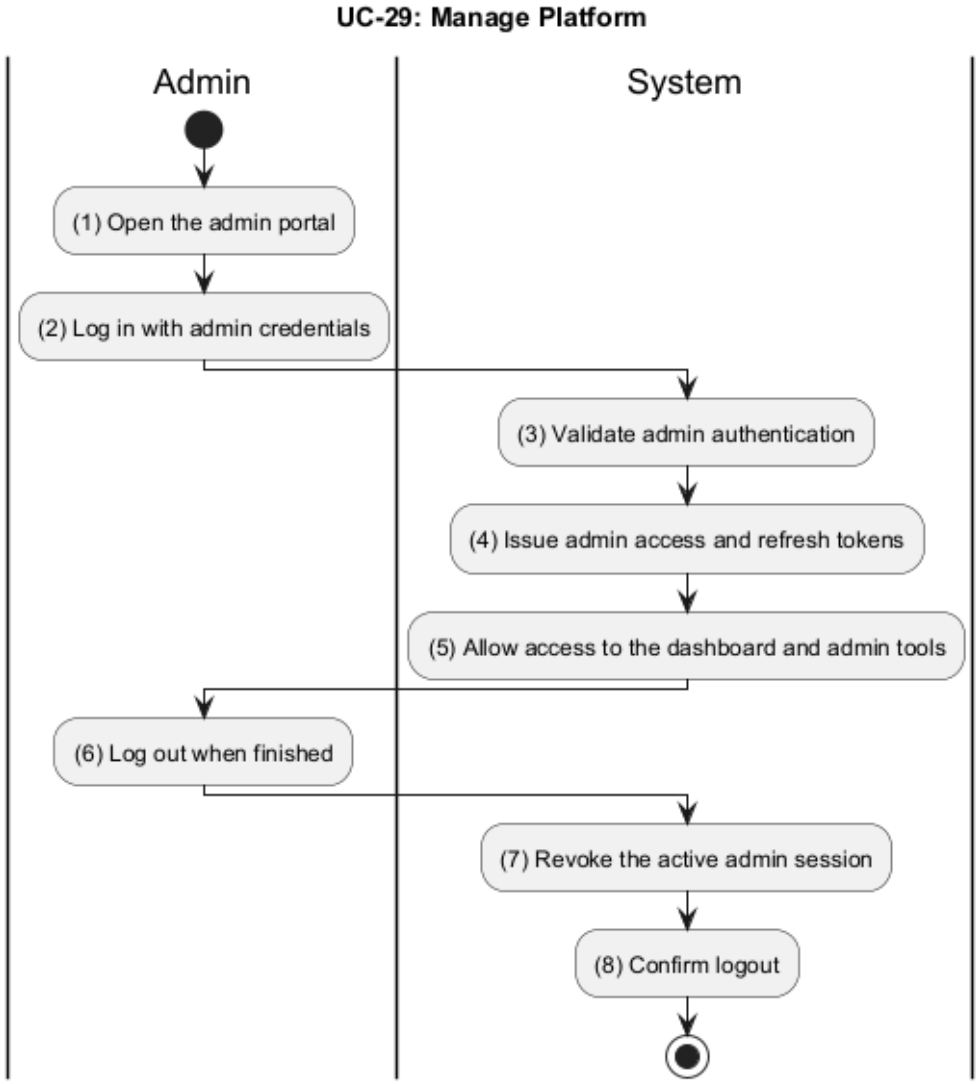


Figure 3.30: Activity Diagram for UC-29

| Activity | BR Code | Description |
|---|---|---|
| (1–5) | BR1 | **Authenticate Admin Access:**<br>• The system checks the admin credentials and requires the user to have the admin role.<br>• The system rejects banned or deleted admin accounts.<br>• The system verifies the password for local accounts and issues access and refresh tokens using ***AdminLogin()***. |

Table 3.57: Business Rules for UC-29 - Manage Platform

| Activity | BR Code | Description |
|---|---|---|
| (6–8) | BR2 | **End the Admin Session:**<br>• The system revokes the active admin session using ***AdminLogout()***.<br>• The system invalidates the stored access and refresh tokens when token revocation is available.<br>• The system confirms logout after session cleanup. |

Table 3.57: Business Rules for UC-29 - Manage Platform

### 3.1.30 UC-30 - View Analytics

| | |
|---|---|
| **Name** | View Analytics |
| **Description** | This use case describes how a system admin reviews platform usage and performance metrics. |
| **Actor** | System Admin |
| **Trigger** | When the admin opens the analytics dashboard. |
| **Pre-condition** | The admin is logged in to the system.<br>Analytics data is available for the selected period. |
| **Post-condition** | The analytics dashboard is displayed with current metrics.<br>The admin can review user, content, and moderation statistics. |

Table 3.58: Use Case Specification - View Analytics

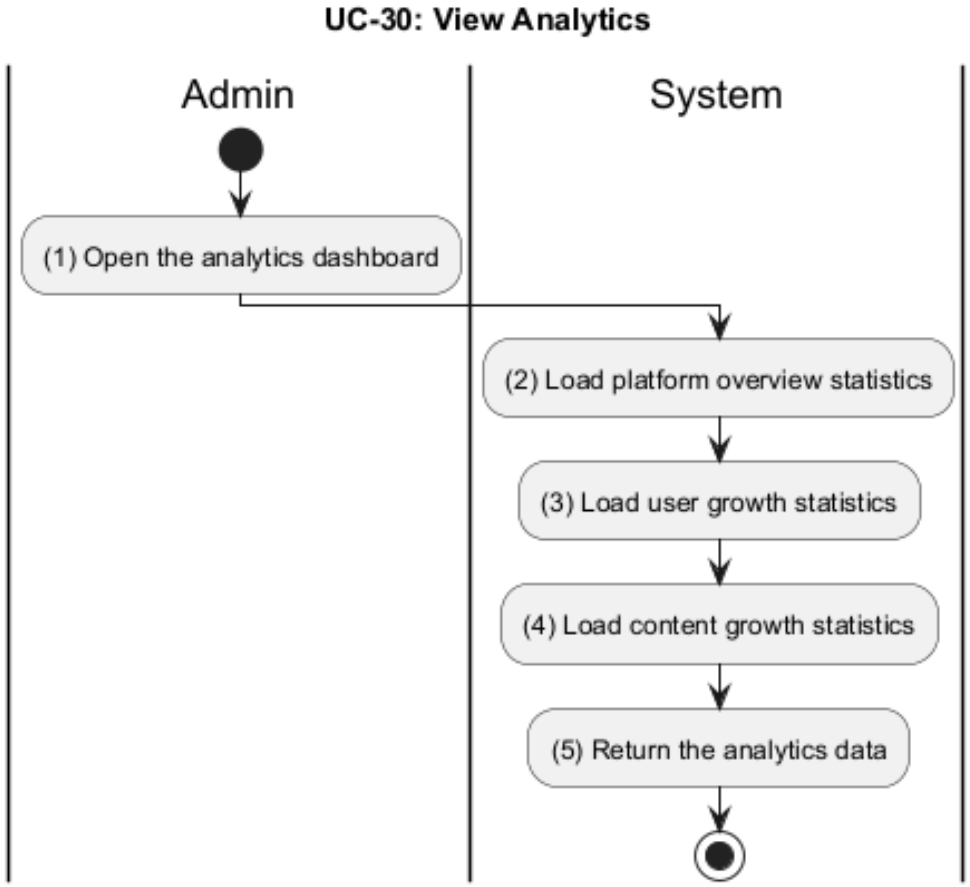


Figure 3.31: Activity Diagram for UC-30

| Activity | BR Code | Description |
|---|---|---|
| (1–5) | BR1 | **Load Platform Analytics:**<br>• The system checks that the requester is an admin.<br>• The system loads platform overview data using ***GetPlatformOverview()***.<br>• The system aggregates user, content, and moderation statistics from the backend services. |
| (2–5) | BR2 | **Return Analytics Breakdowns:**<br>• The system returns user growth and content growth data using ***GetUserStats()*** and ***GetContentStats()***.<br>• The system includes the current overview snapshot in the analytics response. |

Table 3.59: Business Rules for UC-30 - View Analytics

## 3.2 Data Requirements

### 3.2.1 Logical Data Model

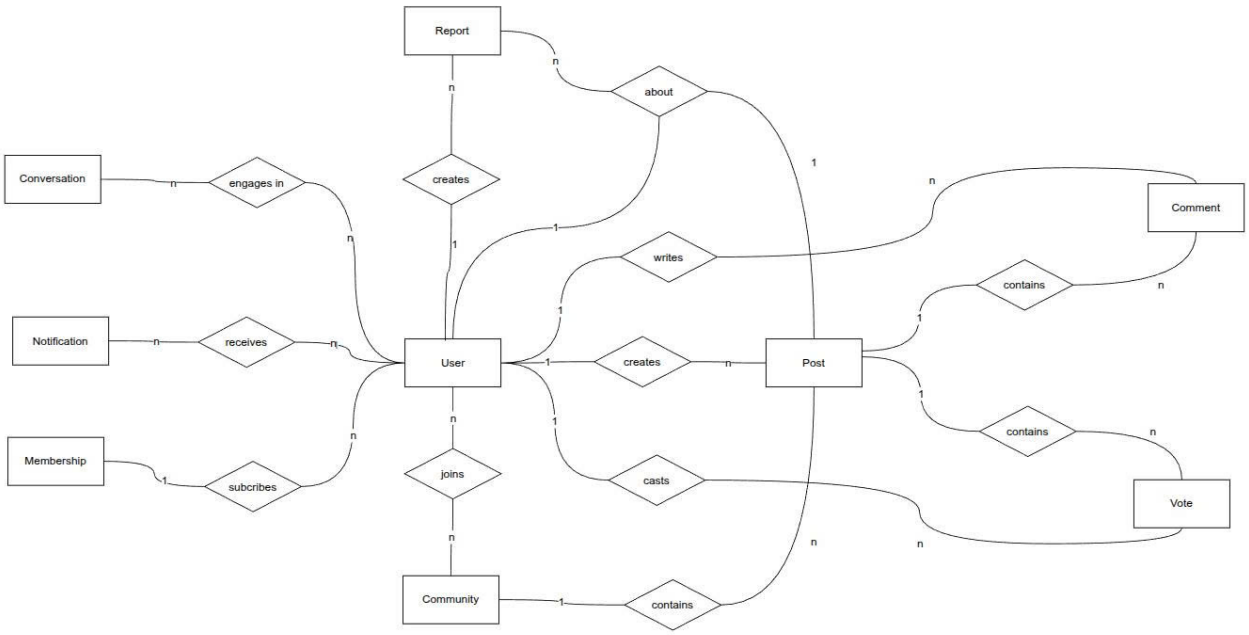


Figure 3.32: Logical Data Model for LKForum

### 3.2.2 Data Dictionary

#### Authentication and User Profile Models

| Data Element | Description | Composition / Data Type | Length | Values |
|---|---|---|---|---|
| ID | Unique identifier of a user. | objectId | 24 | System-generated, unique. |
| Username | Display name used to identify a user account. | string | – | Must be unique within the platform. |
| Email | Email address used for authentication and contact. | string | – | Must be unique and valid. |
| Reputation | User reputation score. | integer | – | System-maintained counter. |

Table 3.60: User

| | | | | |
|---|---|---|---|---|
| Password | Hashed password for a local account. | string | – | Stored only for local providers. |
| Provider | Authentication source for a user account. | enum | – | local, google |
| ProviderID | External account identifier returned by the provider. | string | – | Used for third-party login linkage. |
| Role | Platform-wide role of the user. | enum | – | user, admin |
| RoleContent | Role-specific data attached to the user account. | object | – | Contains user or admin sub-document data. |
| Settings | User preference configuration. | object | – | Contains appearance, notification, privacy, and content settings. |
| IsVerified | Indicates whether the account has been verified. | boolean | – | true, false |
| IsBanned | Indicates whether the account is currently banned. | boolean | – | true, false |
| BanUntil | Timestamp when a ban expires, if any. | timestamp | – | Null means permanent ban. |
| BanReason | Reason recorded for a ban action. | string | – | Optional free text. |
| CreatedAt | Timestamp when the record was created. | timestamp | – | System-generated. |
| DeletedAt | Timestamp for soft-deleted records. | timestamp | – | Null when not deleted. |

Table 3.60: User

| **Data Element** | **Description** | **Composition / Data Type** | **Length** | **Values** |
|---|---|---|---|---|
| AsUser | User-specific profile data. | object | – | See UserRoleContent table. |
| AsAdmin | Admin-specific profile data. | object | – | See AdminRoleContent table. |

Table 3.61: RoleContent

| **Data Element** | **Description** | **Composition / Data Type** | **Length** | **Values** |
|---|---|---|---|---|

Table 3.62: UserRoleContent

| | | | | |
|---|---|---|---|---|
| Avatar | Profile image. | object | – | See Image table. |
| Cover | Cover image. | object | – | See Image table. |
| Bio | Short biography. | string | – | User-generated text. |
| Gender | Declared gender. | enum | – | male, female, prefer_not_to_say |
| DateOfBirth | User date of birth. | timestamp | – | Optional date value. |
| Location | Province or city. | enum | – | One of the provinces defined in model/location.go. |
| Interests | List of user interests. | array[string] | – | Values defined in model/interest.go. |
| SocialLinks | Collection of public social profiles. | object | – | Website, Facebook, YouTube, GitHub links. |
| Stats | User activity statistics. | object | – | Post, comment, and vote counters plus timestamps. |

Table 3.62: UserRoleContent

| **Data Element** | **Description** | **Composition / Data Type** | **Length** | **Values** |
|---|---|---|---|---|
| Website | Personal website URL. | string | – | Optional URL. |
| Facebook | Facebook profile URL. | string | – | Optional URL. |
| YouTube | YouTube channel URL. | string | – | Optional URL. |
| GitHub | GitHub profile URL. | string | – | Optional URL. |

Table 3.63: SocialLinks

| **Data Element** | **Description** | **Composition / Data Type** | **Length** | **Values** |
|---|---|---|---|---|
| PostCount | Number of posts created. | integer | – | System-maintained counter. |
| CommentCount | Number of comments created. | integer | – | System-maintained counter. |
| TotalUpvotes | Total upvotes received. | integer | – | System-maintained counter. |
| JoinedAt | Timestamp when the user joined. | timestamp | – | System-generated. |
| LastActiveAt | Timestamp of the last activity. | timestamp | – | System-generated. |

Table 3.64: ActivityStats

| Data Element | Description | Composition / Data Type | Length | Values |
|---|---|---|---|---|
| Permissions | Admin capability list. | array[string] | – | Permission names granted to the admin. |

Table 3.65: AdminRoleContent

| Data Element | Description | Composition / Data Type | Length | Values |
|---|---|---|---|---|
| Appearance | Visual preferences. | object | – | See AppearanceSettings table. |
| Notifications | Notification preferences. | object | – | See NotificationSettings table. |
| Privacy | Privacy preferences. | object | – | See PrivacySettings table. |
| Content | Content filtering preferences. | object | – | See ContentSettings table. |

Table 3.66: UserSettings

| Data Element | Description | Composition / Data Type | Length | Values |
|---|---|---|---|---|
| Theme | UI theme preference. | enum | – | light, dark, auto |
| FontSize | UI font size preference. | enum | – | small, medium, large |

Table 3.67: AppearanceSettings

| Data Element | Description | Composition / Data Type | Length | Values |
|---|---|---|---|---|
| InAppEnabled | In-app notification toggle. | boolean | – | true, false |
| EmailEnabled | Email notification toggle. | boolean | – | true, false |
| NotifyOn-Comment | Notify on comments. | boolean | – | true, false |
| NotifyOn-Mention | Notify on mentions. | boolean | – | true, false |
| NotifyOnUpvote | Notify on upvotes. | boolean | – | true, false |
| NotifyOnMes-sage | Notify on private messages. | boolean | – | true, false |

Table 3.68: NotificationSettings

| Data Element | Description | Composition / Data Type | Length | Values |
|---|---|---|---|---|
| ShowProfile | Controls whether a profile is public. | boolean | – | true, false |
| ShowEmail | Controls whether email is visible on the profile. | boolean | – | true, false |
| ShowPostHistory | Controls whether post history is visible. | boolean | – | true, false |
| AllowDirectMessages | Controls whether direct messages are allowed. | boolean | – | true, false |
| AllowMentions | Controls whether mentions are allowed. | boolean | – | true, false |

Table 3.69: PrivacySettings

| Data Element | Description | Composition / Data Type | Length | Values |
|---|---|---|---|---|
| AllowNSFW | Controls whether NSFW content is allowed. | boolean | – | true, false |

Table 3.70: ContentSettings

| Data Element | Description | Composition / Data Type | Length | Values |
|---|---|---|---|---|
| EmailVerification.ID | Unique identifier of an email verification record. | objectId | 24 | System-generated, unique. |
| EmailVerification.Email | Email being verified. | string | – | Must be unique and valid. |
| EmailVerification.OTP | One-time password used for verification. | string | 6 | Six-digit numeric code. |
| EmailVerification.OTPExpiresAt | Expiration timestamp for the OTP. | timestamp | – | System-generated. |
| EmailVerification.IsVerified | Indicates whether the OTP has already been verified. | boolean | – | true, false |
| EmailVerification.Nonce | Anti-replay token. | string | – | System-generated random value. |
| EmailVerification.CreatedAt | Timestamp when the record was created. | timestamp | – | System-generated. |

Table 3.71: Verification Records

| Data Element | Description | Composition / Data Type | Length | Values |
|---|---|---|---|---|
| PasswordReset-.ID | Unique identifier of a password reset record. | objectId | 24 | System-generated, unique. |
| PasswordReset-.Email | Email being reset. | string | – | Must be unique and valid. |
| PasswordReset-.OTP | One-time password used for reset. | string | 6 | Six-digit numeric code. |
| PasswordReset-.OTPExpiresAt | Expiration timestamp for the OTP. | timestamp | – | System-generated. |
| PasswordReset-.IsVerified | Indicates whether the OTP has already been verified. | boolean | – | true, false |
| PasswordReset-.Nonce | Anti-replay token. | string | – | System-generated random value. |
| PasswordReset-.CreatedAt | Timestamp when the record was created. | timestamp | – | System-generated. |

Table 3.72: PasswordReset

| Data Element | Description | Composition / Data Type | Length | Values |
|---|---|---|---|---|
| URL | Public URL of a stored image. | string | – | Cloudinary or equivalent media URL. |
| PublicID | Provider-specific image identifier. | string | – | Used for delete/update operations. |
| UploadedAt | Timestamp when the image was uploaded. | timestamp | – | System-generated. |

Table 3.73: Image

| Data Element | Description | Composition / Data Type | Length | Values |
|---|---|---|---|---|
| URL | Public URL of a stored video. | string | – | Cloudinary or equivalent media URL. |
| PublicID | Provider-specific video identifier. | string | – | Used for delete/update operations. |
| UploadedAt | Timestamp when the video was uploaded. | timestamp | – | System-generated. |

Table 3.74: Video

**Community and Membership Models**

| Data Element | Description | Composition / Data Type | Length | Values |
|---|---|---|---|---|
| ID | Unique identifier of a community. | objectId | 24 | System-generated, unique. |
| Name | Public name of a community. | string | – | Must be unique. |
| Description | Text describing a community. | string | – | Optional free text. |
| Avatar | Community avatar image URL. | string | – | Optional media URL. |
| Banner | Banner image for a community. | string | – | Optional media URL. |
| Setting | Community configuration settings. | object | – | Contains privacy and moderation flags. |
| Rules | Published rules of a community. | array[object] | – | Title and description entries. |
| Moderators | List of assigned community moderators. | array[object] | – | Moderator sub-documents. |
| MemberCount | Current number of community members. | integer | – | System-maintained counter. |
| PostCount | Current number of community posts. | integer | – | System-maintained counter. |
| CreateAt | Timestamp when the community was created. | timestamp | – | System-generated. |
| CreateByID | Identifier of the creator. | objectId | 24 | Must correspond to an existing user id. |
| CreateByName | Username of the creator. | string | – | Copied from the creator account. |
| CreateByAvatar | Avatar URL of the creator. | string | – | Copied from the creator profile. |
| Is18Plus | Marks the community as adult content. | boolean | – | true, false |
| IsDeleted | Indicates whether the community is soft deleted. | boolean | – | true, false |
| IsBanned | Indicates whether the community is banned. | boolean | – | true, false |
| BanReason | Reason recorded for a community ban. | string | – | Optional free text. |

Table 3.75: Community

| Data Element | Description | Composition / Data Type | Length | Values |
|---|---|---|---|---|
| IsPrivate | Controls whether the community is visible only to members. | boolean | – | true, false |
| PostRequire-Approval | Controls whether new posts require moderator approval. | boolean | – | true, false |
| JoinRequire-Approval | Controls whether join requests require approval. | boolean | – | true, false |
| MaxPostLength | Maximum allowed post length in characters. | integer | – | Positive integer. |

Table 3.76: CommunitySetting

| Data Element | Description | Composition / Data Type | Length | Values |
|---|---|---|---|---|
| Title | Short title for a community rule. | string | – | User-defined label. |
| Description | Detailed explanation of a community rule. | string | – | User-defined text. |

Table 3.77: CommunityRule

| Data Element | Description | Composition / Data Type | Length | Values |
|---|---|---|---|---|
| UserID | Identifier of the moderator account. | objectId | 24 | Must correspond to an existing user id. |
| Username | Moderator username. | string | – | Copied from the user account. |
| Avatar | Moderator avatar. | object | – | See Image table. |
| IsActive | Indicates whether the moderator role is active. | boolean | – | true, false |
| AssignedAt | Timestamp when the moderator role was assigned. | timestamp | – | System-generated. |

Table 3.78: Moderator

| Data Element | Description | Composition / Data Type | Length | Values |
|---|---|---|---|---|

Table 3.79: Membership

| | | | | |
|---|---|---|---|---|
| ID | Unique identifier of a membership record. | objectId | 24 | System-generated, unique. |
| UserID | Identifier of the member user. | objectId | 24 | Must correspond to an existing user id. |
| CommunityID | Identifier of the target community. | objectId | 24 | Must correspond to an existing community id. |
| Status | Membership review status. | enum | – | pending, approved, rejected |
| CreatedAt | Timestamp when the membership was created. | timestamp | – | System-generated. |
| User | Cached user summary for a membership entry. | object | – | Contains id, username, and avatar. |

Table 3.79: Membership

| **Data Element** | **Description** | **Composition / Data Type** | **Length** | **Values** |
|---|---|---|---|---|
| ID | Identifier of the membership user summary. | objectId | 24 | Must correspond to an existing user id. |
| Username | Username of the membership user summary. | string | – | Copied from the user account. |
| Avatar | Avatar of the membership user summary. | object | – | See Image table. |

Table 3.80: UserInfo

| **Data Element** | **Description** | **Composition / Data Type** | **Length** | **Values** |
|---|---|---|---|---|
| CommunityID | Identifier of the banned community. | objectId | 24 | Must correspond to an existing community id. |
| UserID | Identifier of the banned user. | objectId | 24 | Must correspond to an existing user id. |
| Type | Ban category applied by the community. | enum | – | banned, muted |
| Reason | Reason recorded for the ban or mute. | string | – | Optional free text. |
| BannedAt | Timestamp when the restriction was applied. | timestamp | – | System-generated. |

Table 3.81: CommunityBan

| | | | | |
|---|---|---|---|---|
| BannedBy | Identifier of the moderator who applied the restriction. | objectId | 24 | Must correspond to an existing user id. |
| ExpiresAt | Timestamp when the restriction expires. | timestamp | – | Null for permanent restrictions. |

Table 3.81: CommunityBan

### Content and Moderation Models

| **Data Element** | **Description** | **Composition / Data Type** | **Length** | **Values** |
|---|---|---|---|---|
| ID | Unique identifier of a post. | objectId | 24 | System-generated, unique. |
| AuthorID | Identifier of the post author. | objectId | 24 | Must correspond to an existing user id. |
| CommunityID | Identifier of the community that owns the post. | objectId | 24 | Must correspond to an existing community id. |
| Type | Type of post content. | enum | – | text, poll, video, image |
| Title | Title of a post. | string | – | Text entered by the author. |
| Content | Main post payload. | object | – | Contains text, images, videos, or poll data. |
| VotesCount | Vote summary for a post or comment. | object | – | Up and down counts. |
| CommentsCount | Number of comments on a post. | integer | – | System-maintained counter. |
| HotScore | Ranking score used for best sorting. | float | – | Calculated from votes and time decay. |
| CreatedAt | Timestamp when the post was created. | timestamp | – | System-generated. |
| UpdatedAt | Timestamp of the last content update. | timestamp | – | Null until updated. |
| IsDeleted | Indicates whether the record is soft deleted. | boolean | – | true, false |
| IsHidden | Indicates whether the post is hidden by its author. | boolean | – | true, false |
| IsEdited | Indicates whether the post has been edited. | boolean | – | true, false |
| Tags | Tags attached to a post. | array[string] | – | Free-form keywords. |

Table 3.82: Post

| | | | | |
|---|---|---|---|---|
| IsDraft | Indicates whether the post is saved as a draft. | boolean | – | true, false |
| IsBan | Indicates whether a post is banned. | boolean | – | true, false |
| BanReason | Reason recorded for a post ban. | string | – | Optional free text. |
| ModerationStatus | Current AI or moderator moderation state. | enum | – | pending, approved, rejected, skipped |
| ModerationResult | Moderation analysis details. | object | – | Contains violation flag, confidence, categories, and reason. |
| ModeratedAt | Timestamp when moderation happened. | timestamp | – | Null until moderated. |

Table 3.82: Post

| **Data Element** | **Description** | **Composition / Data Type** | **Length** | **Values** |
|---|---|---|---|---|
| Text | Text body of a post. | string | – | Optional for non-text posts. |
| Images | Attached images in a post. | array[object] | – | Array of image metadata. |
| Videos | Attached videos in a post. | array[object] | – | Array of video metadata. |
| Poll | Poll payload attached to a post. | object | – | Question, options, votes, expiry, and multi-select flag. |

Table 3.83: PostContent

| **Data Element** | **Description** | **Composition / Data Type** | **Length** | **Values** |
|---|---|---|---|---|
| Question | Question presented to poll voters. | string | – | Poll text. |
| Options | Available poll options. | array[object] | – | Each option has id, text, and votes. |
| TotalVotes | Total number of poll votes. | integer | – | System-maintained counter. |
| ExpiresAt | Poll expiry timestamp. | timestamp | – | Null when the poll never expires. |

Table 3.84: Poll

| | | | | |
|---|---|---|---|---|
| AllowMultiple | Controls whether voters may choose more than one option. | boolean | – | true, false |

Table 3.84: Poll

| **Data Element** | **Description** | **Composition / Data Type** | **Length** | **Values** |
|---|---|---|---|---|
| ID | Identifier of a poll option. | string | – | Unique within the poll. |
| Text | Display text of a poll option. | string | – | Option label. |
| Votes | Vote count for a poll option. | integer | – | System-maintained counter. |

Table 3.85: PollOption

| **Data Element** | **Description** | **Composition / Data Type** | **Length** | **Values** |
|---|---|---|---|---|
| Up | Number of upvotes. | integer | – | System-maintained counter. |
| Down | Number of downvotes. | integer | – | System-maintained counter. |

Table 3.86: VotesCount

| **Data Element** | **Description** | **Composition / Data Type** | **Length** | **Values** |
|---|---|---|---|---|
| IsViolation | Indicates whether the content violates policy. | boolean | – | true, false |
| Confidence | Confidence score returned by moderation. | float | – | 0.0 to 1.0 |
| Categories | Categories detected by moderation. | array[string] | – | Issue labels such as hate speech or violence. |
| Reason | Human-readable moderation explanation. | string | – | Localized explanation text. |
| CheckedText | Indicates whether text was analyzed. | boolean | – | true, false |
| CheckedMedia | Indicates whether media was analyzed. | boolean | – | true, false |

Table 3.87: ModerationResult

| Data Element | Description | Composition / Data Type | Length | Values |
|---|---|---|---|---|
| ID | Unique identifier of a comment. | objectId | 24 | System-generated, unique. |
| Author | Author details attached to a comment. | object | – | Contains id, username, and avatar. |
| PostID | Identifier of the parent post. | objectId | 24 | Must correspond to an existing post id. |
| ParentID | Identifier of the parent comment, if any. | objectId | 24 | Null for top-level comments. |
| Content | Comment body text. | string | – | User-generated text. |
| VotesCount | Vote summary for a comment. | object | – | Up and down counts. |
| CreatedAt | Timestamp when the comment was created. | timestamp | – | System-generated. |
| DeletedAt | Timestamp for soft-deleted comments. | timestamp | – | Null when not deleted. |
| IsDeleted | Indicates whether a comment is soft deleted. | boolean | – | true, false |

Table 3.88: Comment

| Data Element | Description | Composition / Data Type | Length | Values |
|---|---|---|---|---|
| ID | Identifier of the comment author. | objectId | 24 | Must correspond to an existing user id. |
| Username | Username of the comment author. | string | – | Copied from the user account. |
| Avatar | Avatar for the comment author. | object | – | Image metadata. |

Table 3.89: CommentAuthor

| Data Element | Description | Composition / Data Type | Length | Values |
|---|---|---|---|---|
| ID | Unique identifier of a draft. | objectId | 24 | System-generated, unique. |
| AuthorID | Identifier of the draft author. | objectId | 24 | Must correspond to an existing user id. |
| CommunityID | Identifier of the draft community, if any. | string | – | Optional community id string. |

Table 3.90: Draft

| | | | | |
|---|---|---|---|---|
| Type | Post type stored in a draft. | enum | – | text, poll, video, image |
| Title | Draft title. | string | – | Optional until publish time. |
| Content | Draft content payload. | object | – | Same shape as post content. |
| Tags | Tags saved with the draft. | array[string] | – | Optional free-form keywords. |
| CreatedAt | Timestamp when the draft was created. | timestamp | – | System-generated. |
| UpdatedAt | Timestamp when the draft was last updated. | timestamp | – | System-generated. |

Table 3.90: Draft

| **Data Element** | **Description** | **Composition / Data Type** | **Length** | **Values** |
|---|---|---|---|---|
| URL | Public URL of a stored image. | string | – | Cloudinary or equivalent media URL. |
| PublicID | Provider-specific image identifier. | string | – | Used for delete/update operations. |
| UploadedAt | Timestamp when the image was uploaded. | timestamp | – | System-generated. |

Table 3.91: Image

| **Data Element** | **Description** | **Composition / Data Type** | **Length** | **Values** |
|---|---|---|---|---|
| URL | Public URL of a stored video. | string | – | Cloudinary or equivalent media URL. |
| PublicID | Provider-specific video identifier. | string | – | Used for delete/update operations. |
| UploadedAt | Timestamp when the video was uploaded. | timestamp | – | System-generated. |

Table 3.92: Video

#### Engagement and Activity Models

| **Data Element** | **Description** | **Composition / Data Type** | **Length** | **Values** |
|---|---|---|---|---|

Table 3.93: SavedPost

| | | | | |
|---|---|---|---|---|
| ID | Unique identifier of a saved-post record. | objectId | 24 | System-generated, unique. |
| UserID | Identifier of the user who saved the post. | objectId | 24 | Must correspond to an existing user id. |
| PostID | Identifier of the saved post. | objectId | 24 | Must correspond to an existing post id. |
| SavedAt | Timestamp when a post was saved. | timestamp | – | System-generated. |

Table 3.93: SavedPost

| **Data Element** | **Description** | **Composition / Data Type** | **Length** | **Values** |
|---|---|---|---|---|
| ID | Unique identifier of a liked-post record. | objectId | 24 | System-generated, unique. |
| UserID | Identifier of the user who liked the post. | objectId | 24 | Must correspond to an existing user id. |
| PostID | Identifier of the liked post. | objectId | 24 | Must correspond to an existing post id. |
| LikedAt | Timestamp when a post was liked. | timestamp | – | System-generated. |

Table 3.94: LikedPost

| **Data Element** | **Description** | **Composition / Data Type** | **Length** | **Values** |
|---|---|---|---|---|
| ID | Unique identifier of a post history record. | objectId | 24 | System-generated, unique. |
| UserID | Identifier of the user who viewed the post. | objectId | 24 | Must correspond to an existing user id. |
| PostID | Identifier of the viewed post. | objectId | 24 | Must correspond to an existing post id. |
| ViewedAt | Timestamp when a post was viewed. | timestamp | – | System-generated. |

Table 3.95: PostHistory

#### Communication Models

| **Data Element** | **Description** | **Composition / Data Type** | **Length** | **Values** |
|---|---|---|---|---|

Table 3.96: Notification

| | | | | |
|---|---|---|---|---|
| ID | Unique identifier of a notification. | objectId | 24 | System-generated, unique. |
| RecipientID | Identifier of the notification recipient. | objectId | 24 | Must correspond to an existing user id. |
| ActorID | Identifier of the user who triggered the notification. | objectId | 24 | Optional when the system is the actor. |
| Type | Type of notification. | enum | – | comment, like, follow, mention, new_message, system |
| Message | Notification text shown to the recipient. | string | – | System-generated or templated text. |
| Link | URL or route associated with the notification. | string | – | Navigation target. |
| IsRead | Indicates whether a notification has been read. | boolean | – | true, false |
| Metadata | Additional notification payload. | Dictionary | – | Arbitrary key-value data. |
| CreatedAt | Timestamp when the notification was created. | timestamp | – | System-generated. |

Table 3.96: Notification

| **Data Element** | **Description** | **Composition / Data Type** | **Length** | **Values** |
|---|---|---|---|---|
| ID | Unique identifier of a private message channel. | objectId | 24 | System-generated, unique. |
| Members | Channel participants. | array[object] | – | Member summaries. |
| Settings | Per-user channel settings. | array[object] | – | Nickname and notification preferences. |
| Status | Channel lifecycle status. | enum | – | active, block |
| CreatedAt | Timestamp when the channel was created. | timestamp | – | System-generated. |
| UpdatedAt | Timestamp when the channel was last updated. | timestamp | – | System-generated. |

Table 3.97: Channel

| **Data Element** | **Description** | **Composition / Data Type** | **Length** | **Values** |
|---|---|---|---|---|

Table 3.98: ChannelSetting

| | | | | |
|---|---|---|---|---|
| UserID | Identifier of the user owning the channel settings. | objectId | 24 | Must correspond to a channel member. |
| Nickname | Custom nickname for the other user in the channel. | string | – | Optional display label. |
| Notification | Controls notification delivery for the channel. | boolean | – | true, false |
| TypingIndicator | Controls typing indicator visibility. | boolean | – | true, false |
| IsDeleted | Indicates whether the channel setting is hidden. | boolean | – | true, false |

Table 3.98: ChannelSetting

| **Data Element** | **Description** | **Composition / Data Type** | **Length** | **Values** |
|---|---|---|---|---|
| UserID | Identifier of the channel member. | objectId | 24 | Must correspond to an existing user id. |
| Username | Username of the channel member. | string | – | Copied from the user account. |
| Avatar | Avatar URL of the channel member. | string | – | Optional media URL. |

Table 3.99: ChannelMember

| **Data Element** | **Description** | **Composition / Data Type** | **Length** | **Values** |
|---|---|---|---|---|
| ID | Unique identifier of a private message. | objectId | 24 | System-generated, unique. |
| ChannelID | Identifier of the private message channel. | objectId | 24 | Must correspond to an existing channel id. |
| SenderID | Identifier of the message sender. | objectId | 24 | Null for system messages. |
| SenderUsername | Cached sender username. | string | – | Copied from the sender account. |
| Type | Message category. | enum | – | user, system |
| Content | Message body text. | string | – | User-generated or system-generated text. |
| IsSend | Indicates whether the message was delivered successfully. | boolean | – | true, false |

Table 3.100: Message

| | | | | |
|---|---|---|---|---|
| IsDeleted | Indicates whether the message is soft deleted. | boolean | – | true, false |
| DeletedAt | Timestamp when the message was deleted. | timestamp | – | Null when not deleted. |

Table 3.100: Message

#### Reporting and Administration Models

| Data Element | Description | Composition / Data Type | Length | Values |
|---|---|---|---|---|
| ID | Unique identifier of a report. | objectId | 24 | System-generated, unique. |
| ReporterID | Identifier of the user who submitted the report. | objectId | 24 | Must correspond to an existing user id. |
| TargetID | Identifier of the reported user, post, or comment. | objectId | 24 | Must correspond to an existing target id. |
| TargetType | Type of reported target. | enum | – | user, post, comment |
| Reason | Short reason for a report or moderation action. | string | – | Free text provided by the user or moderator. |
| Description | Longer report details or explanation. | string | – | Optional free text. |
| IsDeleted | Indicates whether the report is soft deleted. | boolean | – | true, false |
| DeletedAt | Timestamp for deleted reports. | timestamp | – | Null when not deleted. |
| CreatedAt | Timestamp when the report was created. | timestamp | – | System-generated. |

Table 3.101: Report

#### Enum and Value

| Data Element | Description | Composition / Data Type | Length | Values |
|---|---|---|---|---|
| AuthProvider | User authentication source. | enum | – | local, google |
| Role | Platform-wide user role. | enum | – | user, admin |
| Gender | Declared gender. | enum | – | male, female, prefer_not_to_say |

Table 3.102: Enum and Value Classes

| | | | | |
|---|---|---|---|---|
| VNProvince | User location value. | enum | – | One of the provinces defined in model/location.go. |
| Interest | User interest label. | enum | – | Values defined in model/interest.go. |
| PostType | Post content type. | enum | – | text, poll, video, image |
| Moderation-Status | Moderation state. | enum | – | pending, approved, rejected, skipped |
| Membership-Status | Membership review state. | enum | – | pending, approved, rejected |
| Notification-Type | Notification type. | enum | – | comment, like, follow, mention, new_message, system |
| ChannelStatus | Channel lifecycle status. | enum | – | active, block |
| MessageType | Message category. | enum | – | user, system |
| ReportTarget-Type | Reported target type. | enum | – | user, post, comment |
| VoteType | Vote direction type. | enum | – | up, down |
| VoteTargetType | Vote target type. | enum | – | post, comment |

Table 3.102: Enum and Value Classes

## 3.3 Data Integrity, Retention, and Disposal

**DI-1:** The system shall retain user-deleted content (such as posts and comments) in a deactivated state for 30 days. After this period, the content will be permanently purged from the system. This allows users a grace period to recover accidentally deleted items.

**DI-2:** When a user requests to delete their account, the account shall be immediately deactivated and scheduled for permanent deletion. All personally identifiable information and associated content will be permanently deleted or anonymized after 90 days, unless required to be kept for legal reasons.

**DI-3:** Moderation data, including content reports submitted by users and action logs taken by moderators, shall be retained for one year after the report is closed to monitor for patterns of abuse.

**DI-4:** Core content such as active posts and comments will be retained indefinitely as long as the author's account and the community they belong to exist.

# 4 Other Nonfunctional Requirements

## 4.1 Performance Requirements

## 4.2 Security Requirements

## 4.3 Software Quality Attributes

# 5 Other Requirements